

Random_Explorer

February 15, 2026

1 INF421 — Path Planning Algorithms

Membres

- KAMDEM POUOKAM Ivann Harold — **PC**
- NGANTA Pacifique — **PC**

1.0.1 Installation du projet

```
[1]: !pip install uv
```

'pip' n'est pas reconnu en tant que commande interne
ou externe, un programme exécutable ou un fichier de commandes.

```
[2]: import os

if not os.path.exists("src/random_explorer"):
    !uv pip install git+https://www.github.com/KpihX/random-explorer.git
else:
    !uv pip install -e .
```

Resolved 55 packages in 2.09s

Building random-explorer @
file:///D:/TD%20Polytechni/random-explorer

Built random-explorer @
file:///D:/TD%20Polytechni/random-explorer

Prepared 1 package in 59ms

Uninstalled 1 package in 49ms

warning: Failed to hardlink files; falling back to
full copy. This may lead to degraded performance.

If the cache and target directories are on different filesystems,
hardlinking may not be supported.

If this is intentional, set `export UV\_LINK\_MODE=copy` or use `--link-
mode=copy` to suppress this warning.

Installed 1 package in 292ms

~ random-explorer==0.1.0 (from
file:///D:/TD%20Polytechni/random-explorer)

```
[3]: from dotenv import load_dotenv
load_dotenv()

# Kaggle credentials for Colab
try:
    from google.colab import userdata
    os.environ["KAGGLE_API_TOKEN"] = userdata.get('KAGGLE_API_TOKEN')
except:
    pass
```

1.0.2 Chargement des données

```
[4]: import kagglehub
import matplotlib.pyplot as plt
import numpy as np

# Download all scenarios
SCENARIOS = {}
for i in range(5):
    file = f"scenario{i}.txt"
    path = kagglehub.dataset_download(
        handle="ivannkamdem/random-explorer",
        path=file,
    )
    SCENARIOS[f"scenario{i}"] = path
    print(f"Loaded: {file} -> {path}")

# Default scenario for tests
file_path = SCENARIOS["scenario4"]
```

```
d:\TD Polytechni\random-explorer\.venv\Lib\site-packages\tqdm\auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

```
Warning: Looks like you're using an outdated `kagglehub` version (installed:
0.4.3), please consider upgrading to the latest version (1.0.0)Loaded:
```

```
scenario0.txt ->
```

```
C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-
explorer\versions\1\scenario0.txt
```

```
Loaded: scenario1.txt ->
```

```
C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-
explorer\versions\1\scenario1.txt
```

```
Loaded: scenario2.txt ->
```

```
C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-
explorer\versions\1\scenario2.txt
```

```
Loaded: scenario3.txt ->
```

```
C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-
```

```
explorer\versions\1\scenario3.txt
Loaded: scenario4.txt ->
C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-
explorer\versions\1\scenario4.txt
```

1.1 Partie 3 : Particle Swarm Optimization (PSO)

1.1.1 Question 1 : Initialisation de l'environnement

On implémente une classe `Environment` qui : 1. Parse le fichier de scénario (format décrit dans le README) 2. Valide les données (obstacles dans les limites, positions start/goal valides) 3. Fournit une méthode `is_collision(point)` pour la détection de collision

Choix d'implémentation : - Les obstacles sont stockés comme tuples (x, y, width, height) - La validation vérifie que start/goal ne sont pas dans un obstacle - Les bordures des obstacles sont considérées comme collisions

```
[5]: from random_explorer import Environment

env = Environment(file_path)

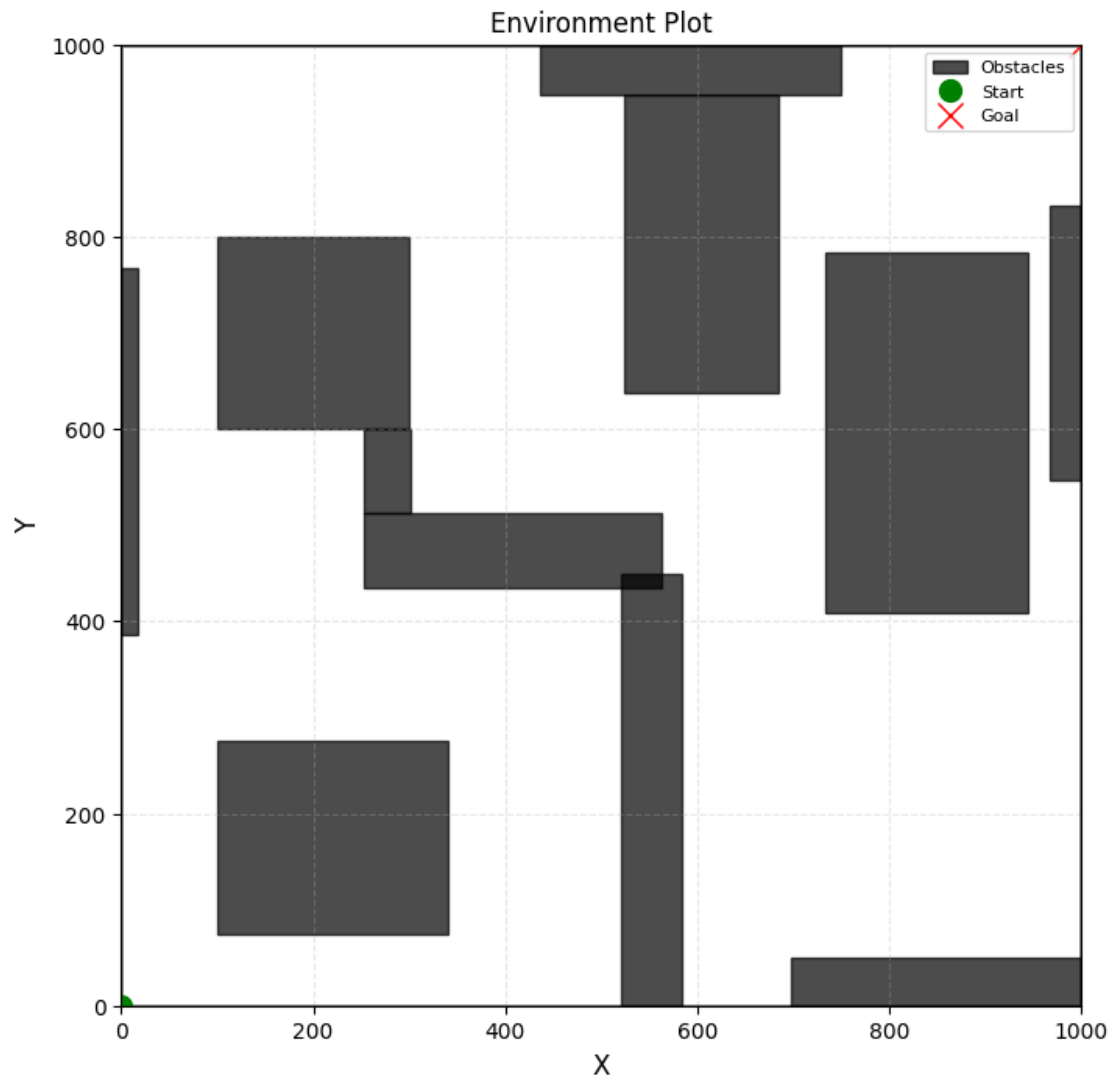
print(f"Environment: {env.x_max} x {env.y_max}")
print(f"Start: {env.start}")
print(f"Goal: {env.goal}")
print(f"Number of obstacles: {len(env.obstacles)}")
```

```
Environment: 1000.0 x 1000.0
Start: (0.0, 0.0)
Goal: (1000.0, 1000.0)
Number of obstacles: 11
```

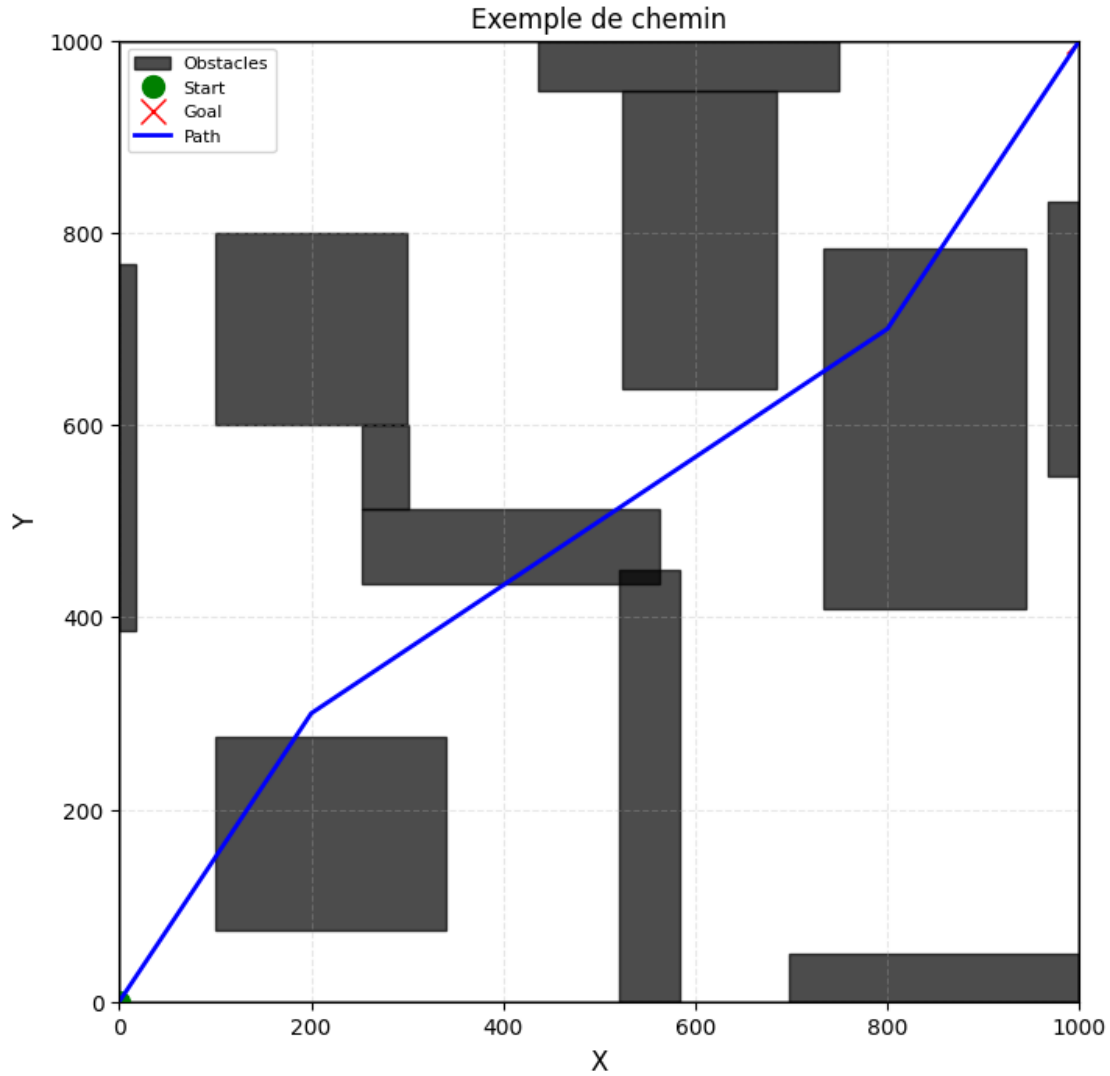
1.1.2 Question 2 : Visualisation de l'environnement

La méthode `plot_environment()` affiche : - Les limites de l'environnement - Les obstacles (rectangles noirs) - Le point de départ (cercle vert) - Le point d'arrivée (croix rouge) - Optionnellement un chemin (ligne bleue)

```
[6]: fig, ax = env.plot_environment()
plt.show()
```



```
[7]: example_path = [(0, 0), (200, 300), (500, 500), (800, 700), (1000, 1000)]
fig, ax = env.plot_environment(path=example_path, title="Exemple de chemin")
plt.show()
```



1.1.3 Question 3 : Définition des particules, fonction de fitness et structure de données

1. Définition d'une particule

Dans le contexte de la planification de trajectoire, une particule représente **un chemin complet candidat**, modélisé par N waypoints intermédiaires :

$$u_s \rightarrow w_1 \rightarrow w_2 \rightarrow \dots \rightarrow w_N \rightarrow u_d$$

2. Fonction de Fitness

$$f(x_i) = \text{Longueur}(x_i) + \text{Pénalité}(x_i)$$

3. Structure de données (NumPy)

- **Positions** X : Matrice $(S, N, 2)$
- **Vitesses** V : Matrice $(S, N, 2)$
- **Meilleurs locaux** P_{best} : Matrice $(S, N, 2)$
- **Meilleur global** G_{best} : Matrice $(N, 2)$

1.1.4 Question 4 : Méthode de détection de collision

Nous avons implémenté deux méthodes de détection de collision, unifiées par une même formule de pénalité :

Formule unifiée :

$$\text{Fitness}(x_i) = L_{path} + P_{base} \cdot \mathbb{1}_{\{C>0\}} + P_{collision} \cdot C$$

Où : - L_{path} = longueur géométrique du chemin - C = métrique de collision (dépend du mode)
- P_{base} = pénalité de base si collision (ex: 1000) - $P_{collision}$ = pénalité par unité de collision (ex: 100)

Mode Hard (échantillonnage) :

Échantillonner p points le long de chaque segment et compter le nombre de segments en collision.

$$C_{hard} = \sum_{j=1}^{N+1} \mathbb{1}_{\{\text{segment}_j \text{ en collision}\}}$$

Simple mais approximatif (dépend de la résolution d'échantillonnage).

Mode Soft (intersection géométrique) :

Calcul exact de la longueur d'intersection segment-obstacle via l'algorithme de clipping de Liang-Barsky.

$$C_{soft} = \sum_{j=1}^{N+1} \sum_{k=1}^M \text{overlap}(\text{segment}_j, \text{obstacle}_k)$$

Plus précis : pénalise proportionnellement à la "profondeur" de pénétration.

1.1.5 Question 5 : Pseudo-code de l'algorithme PSO

Note : L'implémentation supporte les deux modes (`soft_mode=True/False`).

La fonction `Fitness()` utilise la formule unifiée décrite en Q4.

Algorithme PSO_PathPlanning

```
// Initialisation
Pour i = 1..S:
    // Init de chaque particule
    X[i] = N points aléatoires dans [0, x_max] * [0, y_max]
    V[i] = [[0, 0]] * N
    P_best[i], P_best_score[i] = X[i], +∞
    G_best, G_best_score = None, +∞

// Boucle principale
Pour k = 1 à max_iter :
    // Évaluation
    Pour chaque particule i :
        score = Fitness(X[i])
        Si score < P_best_score[i] :
            P_best_score[i] = score
            P_best[i] = copy(X[i])

        Si score < G_best_score :
            G_best_score = score
            G_best = copy(X[i])

    // Mise à jour cinématique
    // r1, r2 = matrices aléatoires de forme (S, N, 2) avec valeurs dans [0, 1]
    r1 = Random(0, 1, taille=(S, N, 2))
    r2 = Random(0, 1, taille=(S, N, 2))
    V = w*V + c1*r1*(P_best - X) + c2*r2*(G_best - X)
    X = Clip(X + V, bornes)

Retourner [Start] + G_best + [Goal]
```

1.1.6 Question 6 : Analyse de complexité

Notations : - K : nombre d'itérations - S : nombre de particules - N : nombre de waypoints par particule (donc $N + 1$ segments) - M : nombre d'obstacles

Mode Soft (intersection géométrique exacte) :

1. Calcul de la longueur du chemin :

- Pour chaque particule : $(N + 1)$ segments $\rightarrow O(N)$ opérations
- Pour toutes les particules : $O(S \cdot N)$

2. Calcul de la pénalité (cœur de la complexité) :

- Pour chaque segment, on calcule l'intersection exacte avec chaque obstacle via intersection paramétrique (clipping de Liang-Barsky) : $O(1)$ par test
- Par segment : $O(M)$ tests
- Par particule ($N + 1$ segments) : $O(N \cdot M)$

- Pour toutes les particules : $O(S \cdot N \cdot M)$
3. **Mise à jour des vitesses et positions :**
- Opérations vectorielles élémentaires sur matrices $(S, N, 2)$: $O(S \cdot N)$

Complexité par itération (soft) : $O(S \cdot N \cdot M)$ (*dominée par l'évaluation fitness*)

Complexité totale (soft) : $O(K \cdot S \cdot N \cdot M)$

Mode Hard (échantillonnage) — en comparaison :

Soit p le nombre de points échantillonnés par segment.

1. **Calcul de la longueur :** identique $\rightarrow O(S \cdot N)$
2. **Calcul de la pénalité :**
 - Pour chaque segment : échantillonner p points et vérifier chaque point contre M obstacles
 - Test point-dans-rectangle : $O(1)$
 - Par segment : $O(p \cdot M)$
 - Par particule : $O(N \cdot p \cdot M)$
 - Pour toutes les particules : $O(S \cdot N \cdot p \cdot M)$

Complexité totale (hard) : $O(K \cdot S \cdot N \cdot p \cdot M)$

Comparaison :

Aspect	Mode Soft	Mode Hard
Complexité	$O(K \cdot S \cdot N \cdot M)$	$O(K \cdot S \cdot N \cdot p \cdot M)$
Facteur supplémentaire	—	$\times p$
Précision	Exacte	Approximative (dépend de p)
Constantes cachées	Plus élevées (calculs géométriques)	Plus faibles

Conclusion : Le mode soft est asymptotiquement plus efficace (p fois moins d'opérations) et plus précis. Le mode hard nécessiterait un p élevé pour être précis, ce qui dégrade ses performances.

1.1.7 Question 7 : Implémentation

```
[8]: # Top 7 configurations from grid search (scenarios 3 & 4)
# Rank 1: 100% success, avg_length=1894.0
pso_params_1 = {'num_particles': 300, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.8, 'c1': 1.6, 'c2': 1.0}
# Rank 2: 100% success, avg_length=2072.3
pso_params_2 = {'num_particles': 250, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.85, 'c1': 1.4, 'c2': 0.8}
# Rank 3: 100% success, avg_length=3714.8
pso_params_3 = {'num_particles': 300, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.85, 'c1': 1.6, 'c2': 1.4}
```

```

# Rank 4: 75% success, avg_length=1848.1
pso_params_4 = {'num_particles': 400, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.7, 'c1': 1.6, 'c2': 0.8}
# Rank 5: 75% success, avg_length=2101.1
pso_params_5 = {'num_particles': 300, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.85, 'c1': 1.0, 'c2': 1.4}
# Rank 6: 75% success, avg_length=2602.8
pso_params_6 = {'num_particles': 400, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.5, 'c1': 1.6, 'c2': 1.6}
# Rank 7: 50% success, avg_length=1551.0
pso_params_7 = {'num_particles': 400, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.7, 'c1': 1.2, 'c2': 1.0}

```

```
[9]: from random_explorer import PSOPathPlanner
```

```

file_path = SCENARIOS["scenario4"]
env = Environment(file_path)

pso_params = pso_params_1.copy()

pso = PSOPathPlanner(env, **pso_params)

path, path_length, score, history = pso.solve(soft_mode=True)
print(f"Score final: {score:.2f}, Longueur: {path_length:.2f}")

```

```
PSO: 100%|          | 100/100 [00:17<00:00, 5.71it/s]
```

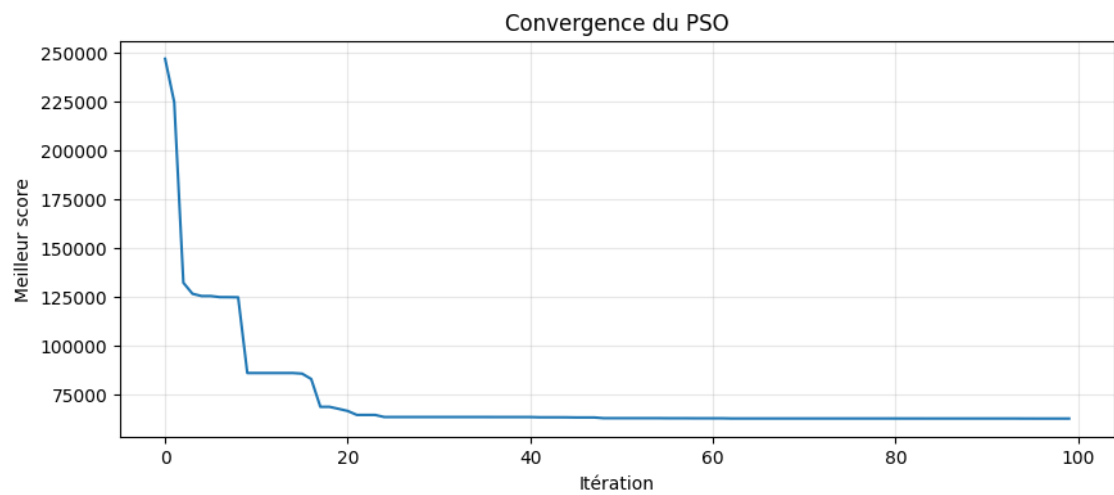
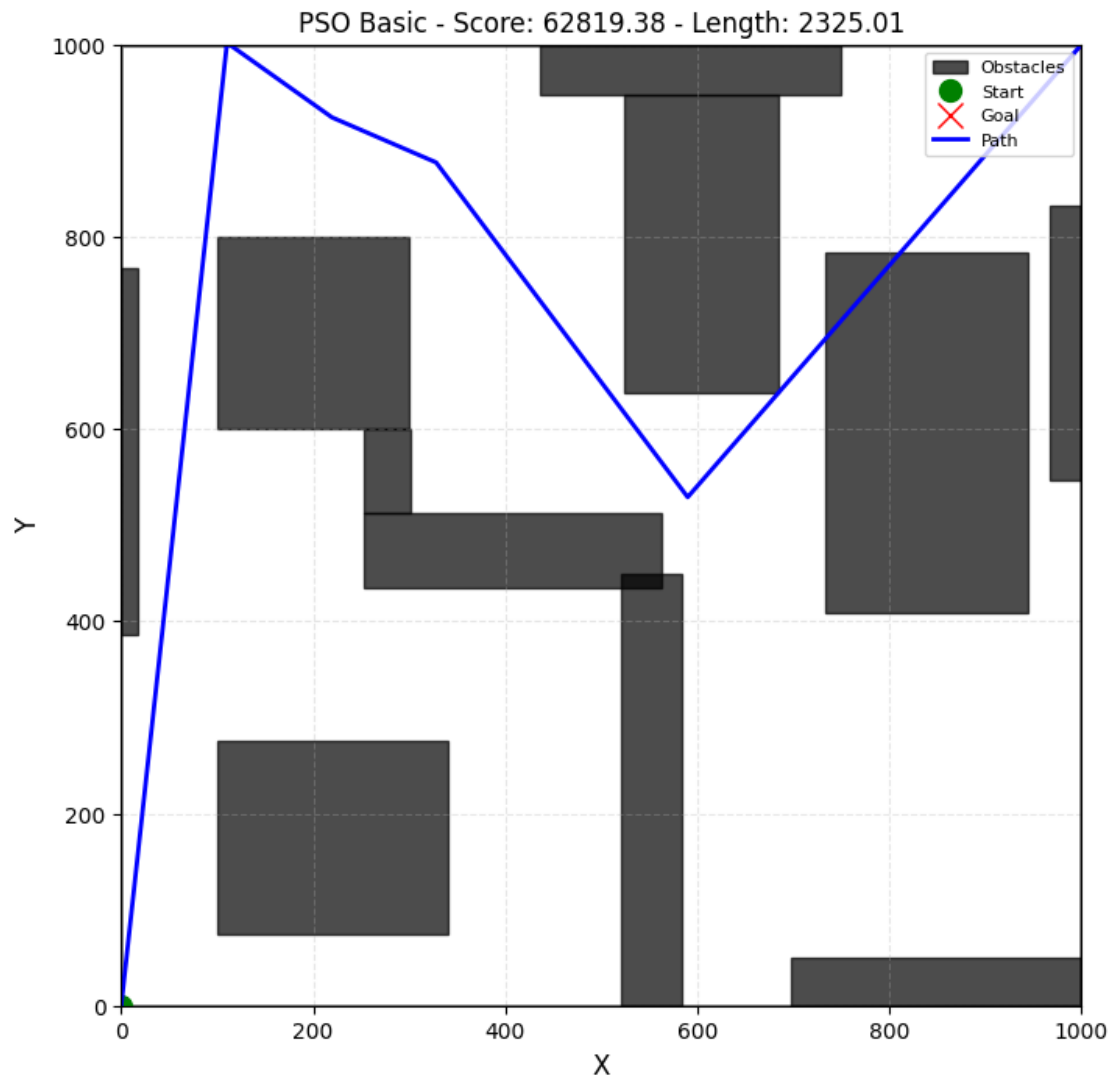
```
Score final: 62819.38, Longueur: 2325.01
```

```
[10]: fig, ax = env.plot_environment(path=path, title=f"PSO Basic - Score: {score:.2f} - Length: {path_length:.2f}")
```

```

plt.figure(figsize=(10, 4))
plt.plot(history)
plt.xlabel('Itération')
plt.ylabel('Meilleur score')
plt.title('Convergence du PSO')
plt.grid(True, alpha=0.3)
plt.show()

```



1.1.8 Question 8 : Performances sur tous les scénarios

Hyperparamètres retenus : Meilleure config du grid search (pso_params_1)

```
[11]: from random_explorer import Benchmark

benchmark = Benchmark()

PSO_PARAMS = pso_params_1.copy() # Best config from grid search

print("Benchmark PSO sur tous les scénarios")
print(f"Params: S={PSO_PARAMS['num_particles']},  

      ↪N={PSO_PARAMS['num_waypoints']}, max_iter={PSO_PARAMS['max_iter']}")
print("="*60)

pso_results = {}
for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)
    result = benchmark.run_pso(env_test, params=PSO_PARAMS)
    result.scenario = name
    pso_results[name] = result

    valid_str = " " if result.is_valid else " "
    print(f"{name}: Length={result.path_length:.2f}, Valid={valid_str},  

      ↪Time={result.cpu_time:.3f}s")
```

Benchmark PSO sur tous les scénarios

Params: S=300, N=8, max_iter=100

=====

PSO: 100%| | 100/100 [00:03<00:00, 26.45it/s]

scenario0: Length=1445.90, Valid= , Time=1.625s

PSO: 100%| | 100/100 [00:06<00:00, 14.29it/s]

scenario1: Length=1965.97, Valid= , Time=3.156s

PSO: 100%| | 100/100 [00:12<00:00, 7.72it/s]

scenario2: Length=1656.70, Valid= , Time=6.438s

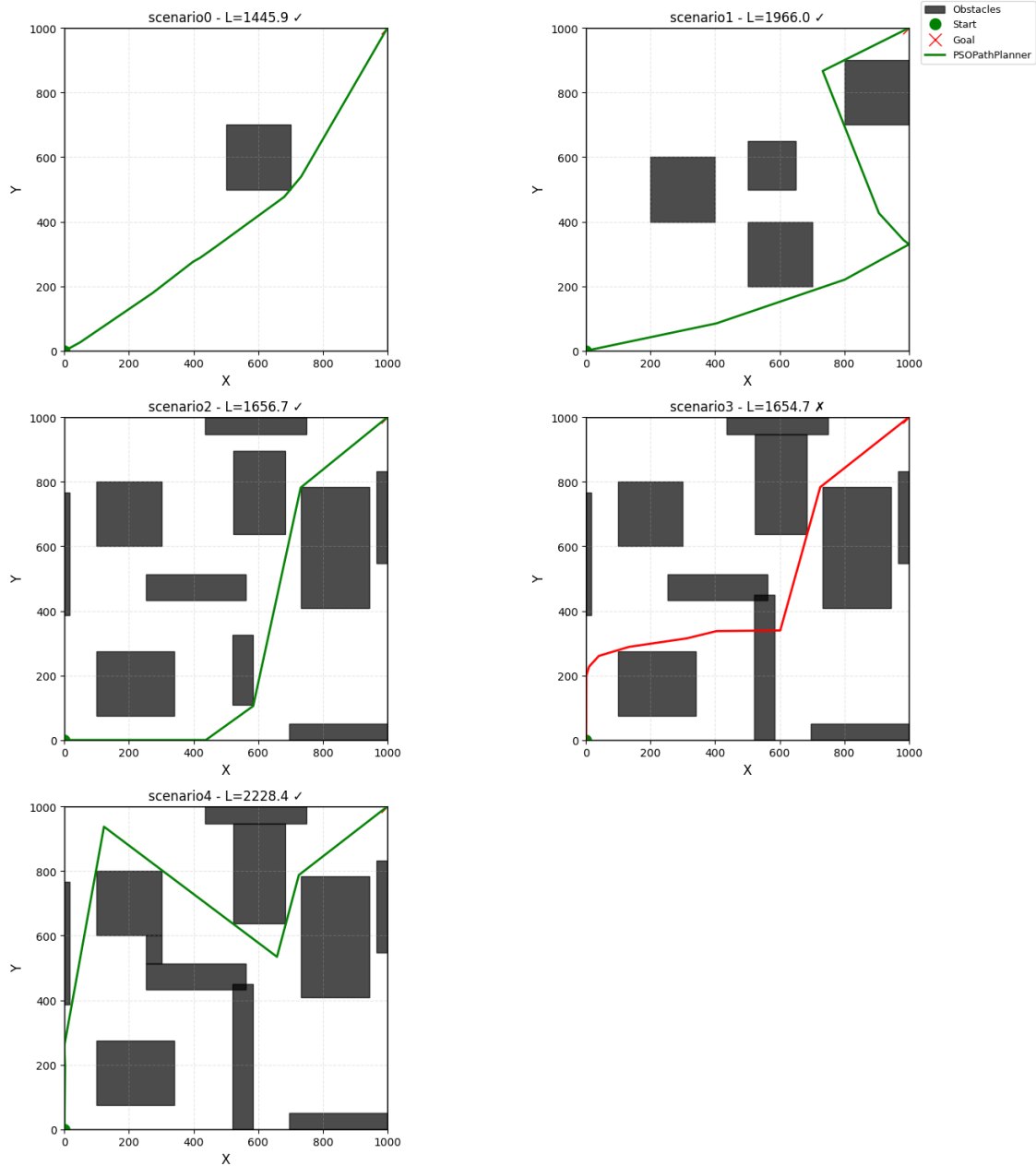
PSO: 100%| | 100/100 [00:19<00:00, 5.25it/s]

scenario3: Length=1654.73, Valid= , Time=13.359s

PSO: 100%| | 100/100 [00:19<00:00, 5.26it/s]

scenario4: Length=2228.36, Valid= , Time=9.594s

```
[12]: # Visualisation des chemins PSO pour tous les scénarios
envs = {name: Environment(path) for name, path in SCENARIOS.items()}
benchmark.plot_results_grid(pso_results, envs, n_cols=2)
plt.show()
```



1.2 Partie 3.2 : Améliorations du PSO

1.2.1 Question 9 : Random Restart

Réinitialise périodiquement les particules (sauf les élites) pour explorer de nouvelles régions.

Hyperparamètres : restart_frequency=50, elite_ratio=0.1

```
[13]: from random_explorer import PSORestart

file_path = SCENARIOS["scenario4"]
env = Environment(file_path)

pso_params = pso_params_1.copy()
pso_params["num_particles"] = 250
pso_params["w"] = 0.85
pso_params["c1"] = 1.6
pso_params["c2"] = 1.1

pso_restart = PSORestart(
    env, **pso_params,
    restart_frequency=50, elite_ratio=0.4
)

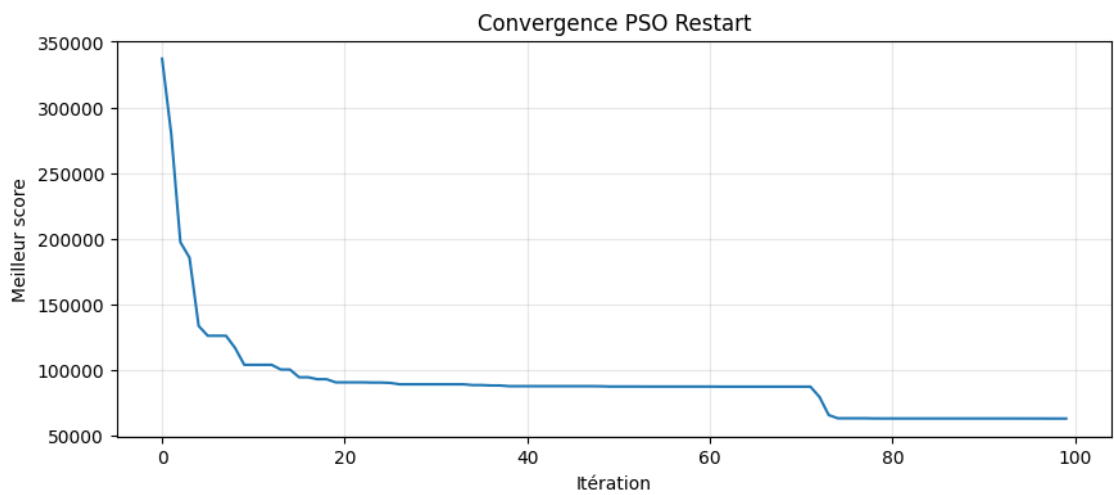
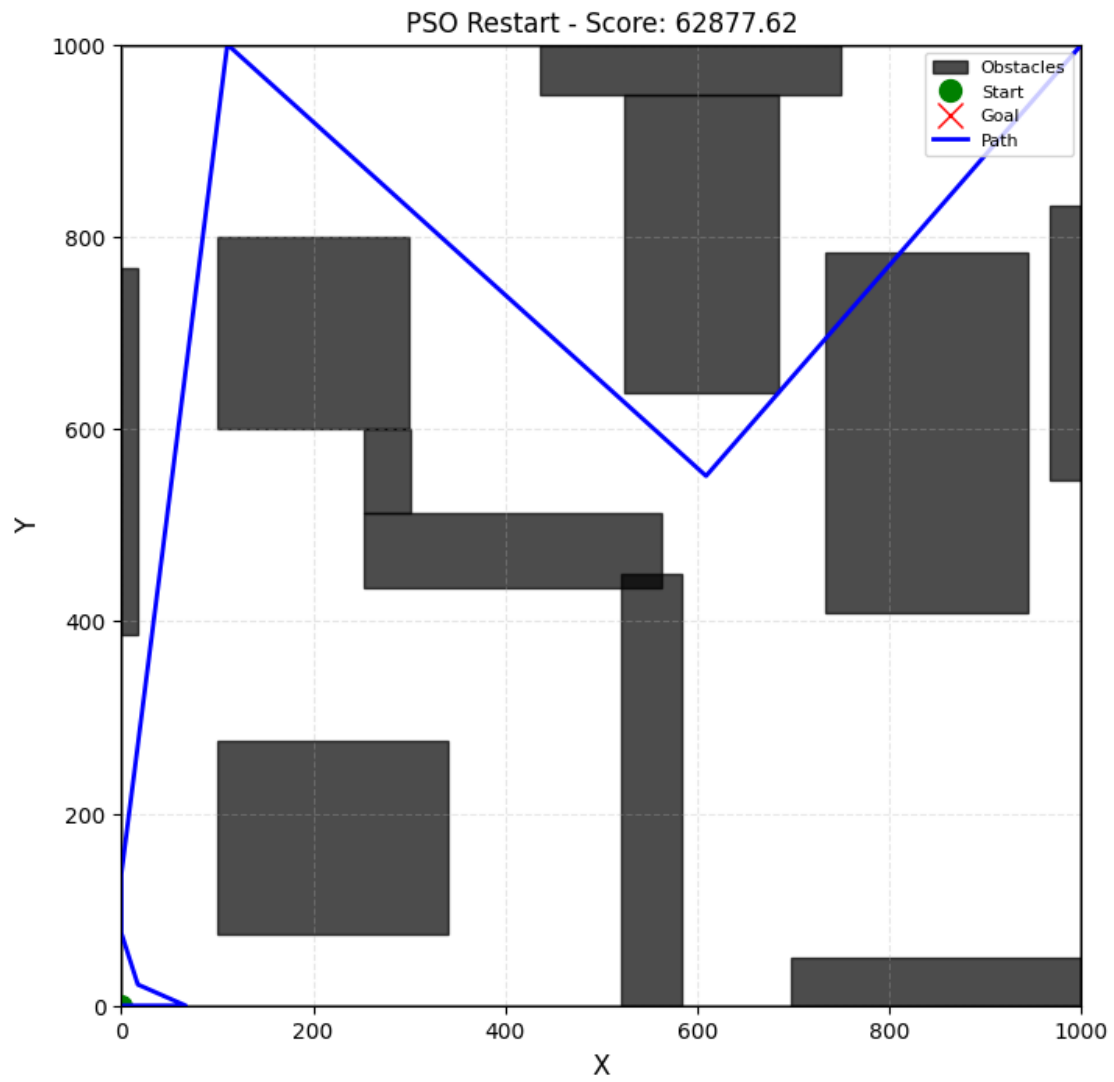
path_restart, length_restart, score_restart, history_restart = pso_restart.
    ↪solve(soft_mode=True)
print(f"PSO Restart - Score: {score_restart:.2f}, Length: {length_restart:.2f},
    ↪Restarts: {pso_restart.num_restarts}")
```

```
PSO-Restart: 100%|          | 100/100 [00:18<00:00,  5.37it/s]
```

```
PSO Restart - Score: 62877.62, Length: 2375.45, Restarts: 1
```

```
[14]: fig, ax = env.plot_environment(path=path_restart, title=f"PSO Restart - Score:
    ↪{score_restart:.2f}")
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(history_restart)
plt.xlabel('Itération')
plt.ylabel('Meilleur score')
plt.title('Convergence PSO Restart')
plt.grid(True, alpha=0.3)
plt.show()
```



1.2.2 Question 10 : Simulated Annealing

Accepte parfois des solutions moins bonnes avec probabilité $p = e^{-\Delta/T}$.

Hyperparamètres : T0=1000, beta=0.95

```
[15]: from random_explorer import PSOSimulatedAnnealing

file_path = SCENARIOS["scenario4"]
env = Environment(file_path)

pso_params = pso_params_1.copy()
pso_params["num_particles"] = 250
pso_params["w"] = 0.85
pso_params["c1"] = 1.6
pso_params["c2"] = 1.0

pso_sa = PSOSimulatedAnnealing(
    env, **pso_params,
    T0=50.0, beta=0.9,
    restart_frequency=50, elite_ratio=0.7
)

path_sa, length_sa, score_sa, history_sa = pso_sa.solve(restart=False,
    ↪soft_mode=True)
print(f"PSO SA - Score: {score_sa:.2f}, Length: {length_sa:.2f}")
```

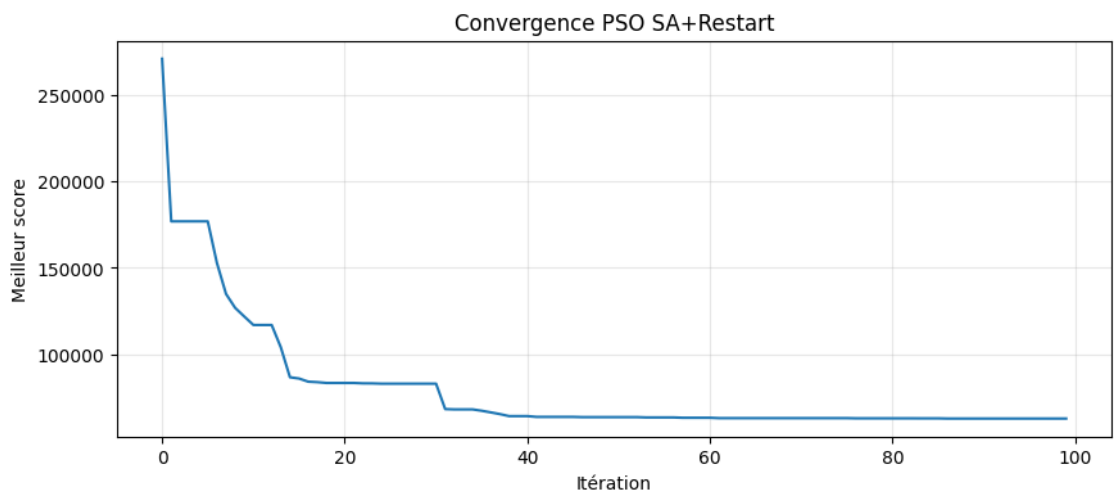
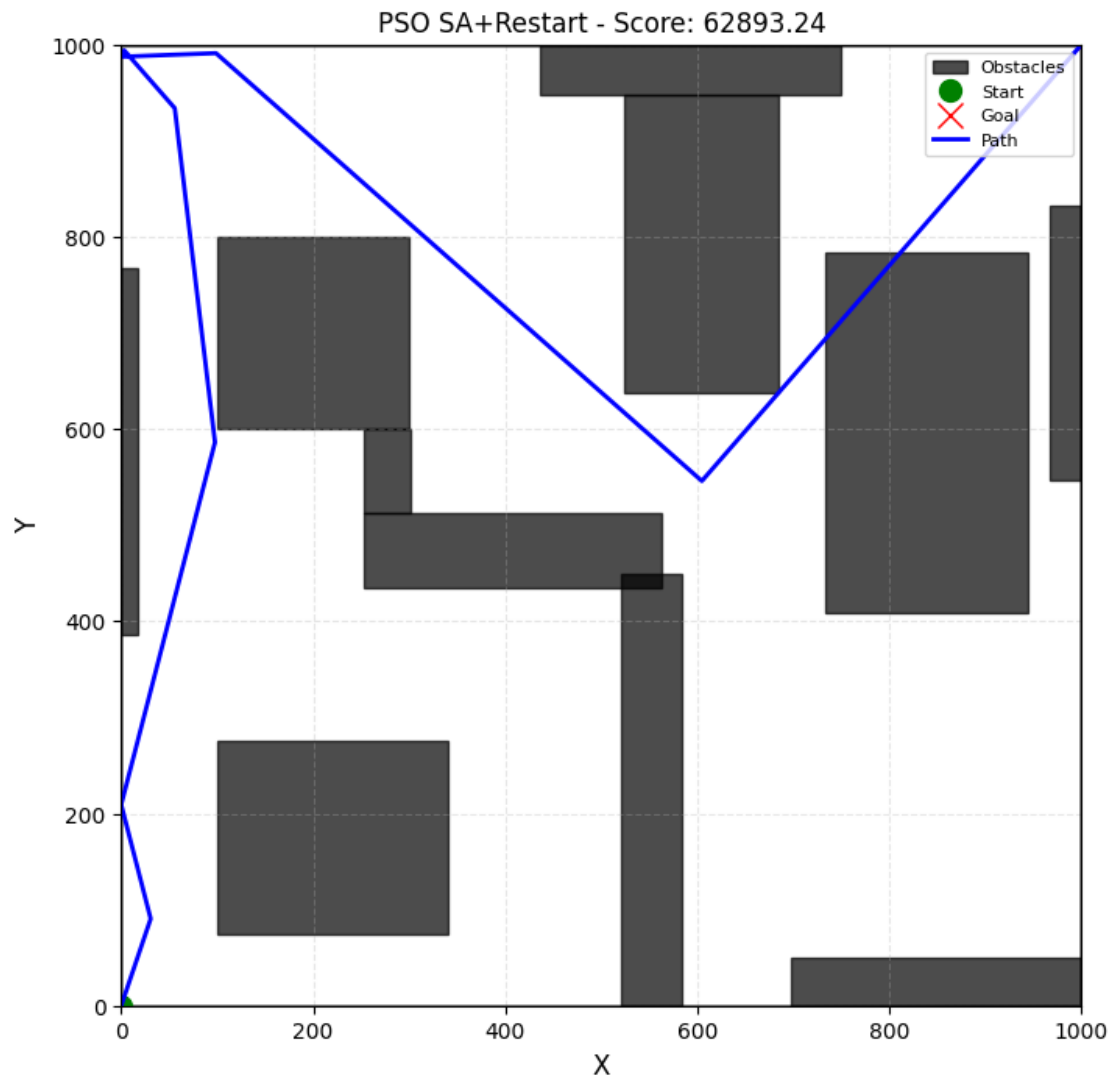
PSO-SA: 0%| | 0/100 [00:00<?, ?it/s]

PSO-SA: 100%| | 100/100 [00:16<00:00, 6.11it/s]

PSO SA - Score: 62893.24, Length: 2419.57

```
[16]: fig, ax = env.plot_environment(path=path_sa, title=f"PSO SA+Restart - Score:␣
    ↪{score_sa:.2f}")
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(history_sa)
plt.xlabel('Itération')
plt.ylabel('Meilleur score')
plt.title('Convergence PSO SA+Restart')
plt.grid(True, alpha=0.3)
plt.show()
```



1.2.3 Question 11 : Dimensional Learning

Si une particule stagne, améliore son P_{best} dimension par dimension depuis G_{best} .

Hyperparamètre : wait_limit=10

```
[17]: from random_explorer import PSODimensionalLearning

file_path = SCENARIOS["scenario4"]
env = Environment(file_path)

pso_params = pso_params_2.copy()
pso_params["num_waypoints"] = 8
pso_params["c2"] = 1.0

pso_dl = PSODimensionalLearning(
    env, **pso_params,
    wait_limit=50,
    T0=100, beta=0.7,
    restart_frequency=50, elite_ratio=0.9
)

path_dl, length_dl, score_dl, history_dl = pso_dl.
    ↪solve(simulated_annealing=False, restart=False, soft_mode=True)
print(f"PSO DL - Score: {score_dl:.2f}, Length: {length_dl:.2f}")
```

PSO-DL: 100%| | 100/100 [00:17<00:00, 5.68it/s]

PSO DL - Score: 169268.58, Length: 3197.59

```
[18]: fig, ax = env.plot_environment(path=path_dl, title=f"PSO DL+SA+Restart - Score:␣
    ↪{score_dl:.2f}")
plt.show()

plt.figure(figsize=(10, 4))
plt.plot(history_dl)
plt.xlabel('Itération')
plt.ylabel('Meilleur score')
plt.title('Convergence PSO DL+SA+Restart')
plt.grid(True, alpha=0.3)
plt.show()
```


1.2.4 Question 12 : Amélioration propre - Adaptive Inertia + Early Stopping

1. Inertie adaptative : Le poids d'inertie décroît linéairement : $w(k) = w_{max} - (w_{max} - w_{min}) \cdot \frac{k}{K}$

Cela favorise l'exploration au début et l'exploitation à la fin.

2. Arrêt anticipé (Early Stopping) : On s'arrête dès qu'un chemin **valide** est trouvé, i.e. quand score = longueur (aucune pénalité de collision).

Justification : - PSO n'offre aucune garantie d'optimalité (le plus court chemin) - Les courbes de convergence montrent de longs paliers (parfois 100+ itérations sans amélioration) - Dès qu'un chemin valide est trouvé, continuer a peu de chances d'améliorer significativement

Paramètres supplémentaires : `dimensional_learning=True/False` pour activer/désactiver le DL.

```
[19]: from random_explorer import PSOAdaptiveInertia

pso_params = pso_params_1.copy()
# pso_params["max_iter"] = 100
pso_params["num_waypoints"] = 9

# Remove 'w' from params since we use w_max/w_min
adaptive_params = {k: v for k, v in pso_params.items() if k != 'w'}

pso_adaptive = PSOAdaptiveInertia(
    env, **adaptive_params,
    w_max=0.9, w_min=0.4,
    wait_limit=25,
    T0=100, beta=0.7,
    restart_frequency=30, elite_ratio=0.5
)

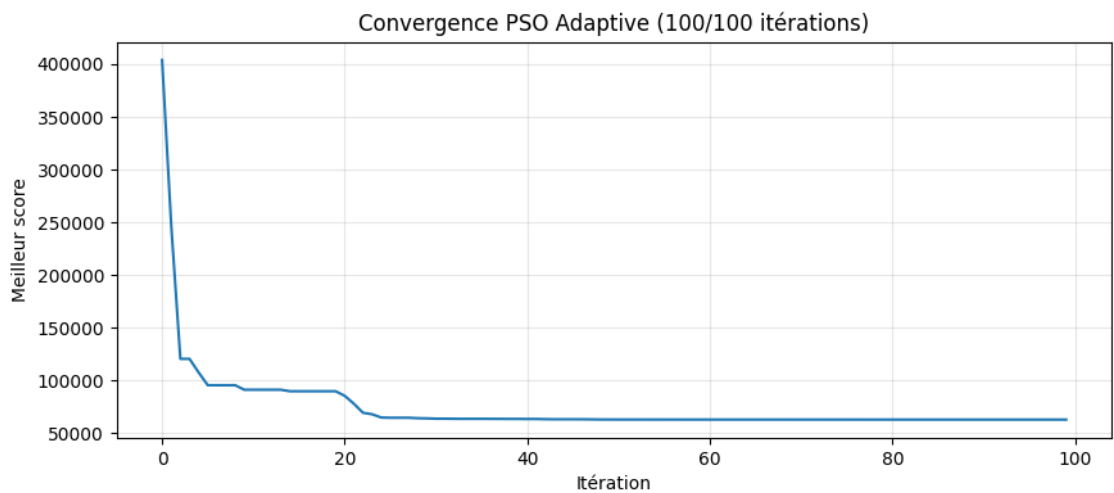
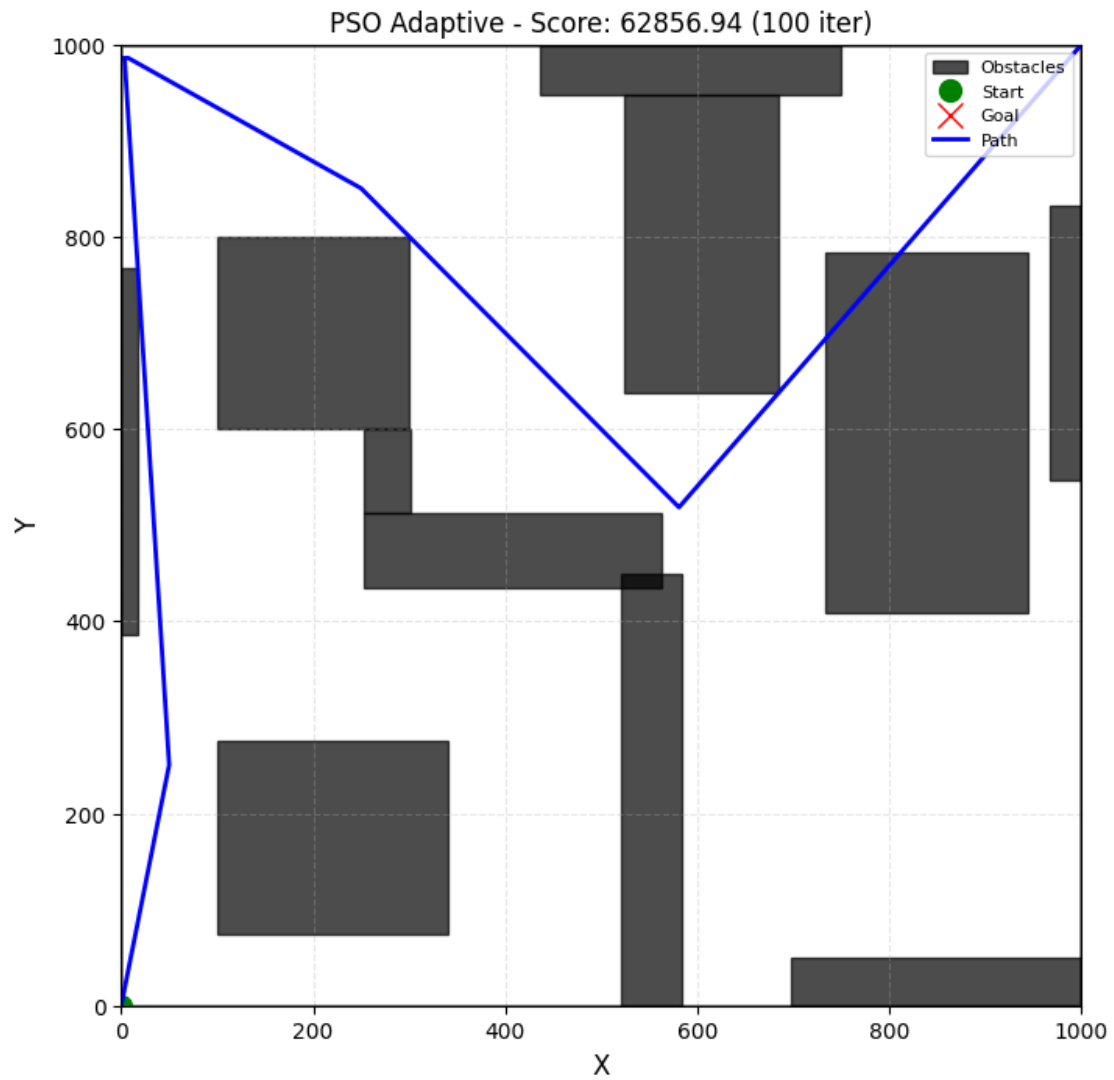
path_adaptive, length_adaptive, score_adaptive, history_adaptive = pso_adaptive.
    ↪solve(
        soft_mode=True,
        restart=False,
        simulated_annealing=False,
        dimensional_learning=False
    )

is_valid = np.isclose(score_adaptive, length_adaptive)
valid_str = " valide" if is_valid else " collision"
print(f"PSO Adaptive - Score: {score_adaptive:.2f}, Length: {length_adaptive:.
    ↪2f}")
print(f"Itérations: {len(history_adaptive)}/{pso_adaptive.max_iter}
    ↪({valid_str})")
```

```
PSO-Adaptive: 100%|      | 100/100 [00:34<00:00, 2.93it/s]
PSO Adaptive - Score: 62856.94, Length: 2385.70
Itérations: 100/100 ( collision)
```

```
[20]: fig, ax = env.plot_environment(path=path_adaptive, title=f"PSO Adaptive - Score:
      ↪ {score_adaptive:.2f} ({len(history_adaptive)} iter)")
      plt.show()

      plt.figure(figsize=(10, 4))
      plt.plot(history_adaptive)
      plt.xlabel('Itération')
      plt.ylabel('Meilleur score')
      plt.title(f'Convergence PSO Adaptive ({len(history_adaptive)}/{pso_adaptive.
      ↪max_iter} itérations)')
      plt.grid(True, alpha=0.3)
      plt.show()
```



```
[21]: # Configuration et benchmark de toutes les variantes PSO sur un scénario
      ↪ajustable
      from random_explorer import Benchmark
      import numpy as np

      scenario = SCENARIOS["scenario2"]
      env = Environment(scenario)

      benchmark = Benchmark()
      pso_params = pso_params_1.copy() # Best config from grid search

      # Run all PSO variants with same base params
      pso_variant_results = benchmark.run_all_pso_variants(env,
      ↪base_params=pso_params)

      print(f"Benchmark de toutes les variantes PSO sur {scenario}")
      print(f"Base params: {pso_params}")
      print("=" * 60)
```

PSO Benchmark

```
Running PSOPathPlanner with params: {'num_particles': 300, 'num_waypoints': 8,
↪'max_iter': 100, 'w': 0.8, 'c1':
1.6, 'c2': 1.0} and solve_kwargs: {'soft_mode': True}
↪
```

PSO: 100%| | 100/100 [00:25<00:00, 3.93it/s]

PSO Benchmark

```
Running PSORestart with params: {'num_particles': 300, 'num_waypoints': 8,
↪'max_iter': 100, 'w': 0.8, 'c1':
1.6, 'c2': 1.0, 'restart_frequency': 50, 'elite_ratio': 0.4} and solve_kwargs:
↪{'soft_mode': True}
```

PSO-Restart: 100%| | 100/100 [00:27<00:00, 3.70it/s]

PSO Benchmark

```
Running PSOSimulatedAnnealing with params: {'num_particles': 300,
↪'num_waypoints': 8, 'max_iter': 100, 'w':
0.8, 'c1': 1.6, 'c2': 1.0, 'T0': 50, 'beta': 0.9, 'restart_frequency': 50,
↪'elite_ratio': 0.7} and
solve_kwargs: {'soft_mode': True, 'restart': False}
↪
```

PSO-SA: 100%| | 100/100 [00:41<00:00, 2.43it/s]

PSO Benchmark

```
Running PSODimensionalLearning with params: {'num_particles': 300,
↪ 'num_waypoints': 8, 'max_iter': 100, 'w':
0.8, 'c1': 1.6, 'c2': 1.0, 'wait_limit': 50, 'T0': 100, 'beta': 0.7,
↪ 'restart_frequency': 50, 'elite_ratio':
0.9} and solve_kwargs: {'soft_mode': True, 'simulated_annealing': False,
↪ 'restart': False}
```

PSO-DL: 100%| | 100/100 [00:50<00:00, 2.00it/s]

PSO Benchmark

```
Running PSOAdaptiveInertia with params: {'num_particles': 300, 'num_waypoints':
↪ 8, 'max_iter': 100, 'c1': 1.6,
'c2': 1.0, 'w_max': 0.9, 'w_min': 0.4, 'wait_limit': 25, 'T0': 100, 'beta': 0.
↪ 7, 'restart_frequency': 30,
'elite_ratio': 0.5} and solve_kwargs: {'soft_mode': True, 'simulated_annealing':
↪ False, 'restart': False,
'dimensional_learning': False}
↪
```

PSO-Adaptive: 25%| | 25/100 [00:11<00:35, 2.10it/s]

Benchmark de toutes les variantes PSO sur

C:\Users\pacifique\.cache\kagglehub\datasets\ivannkamdem\random-explorer\versions\1\scenario2.txt

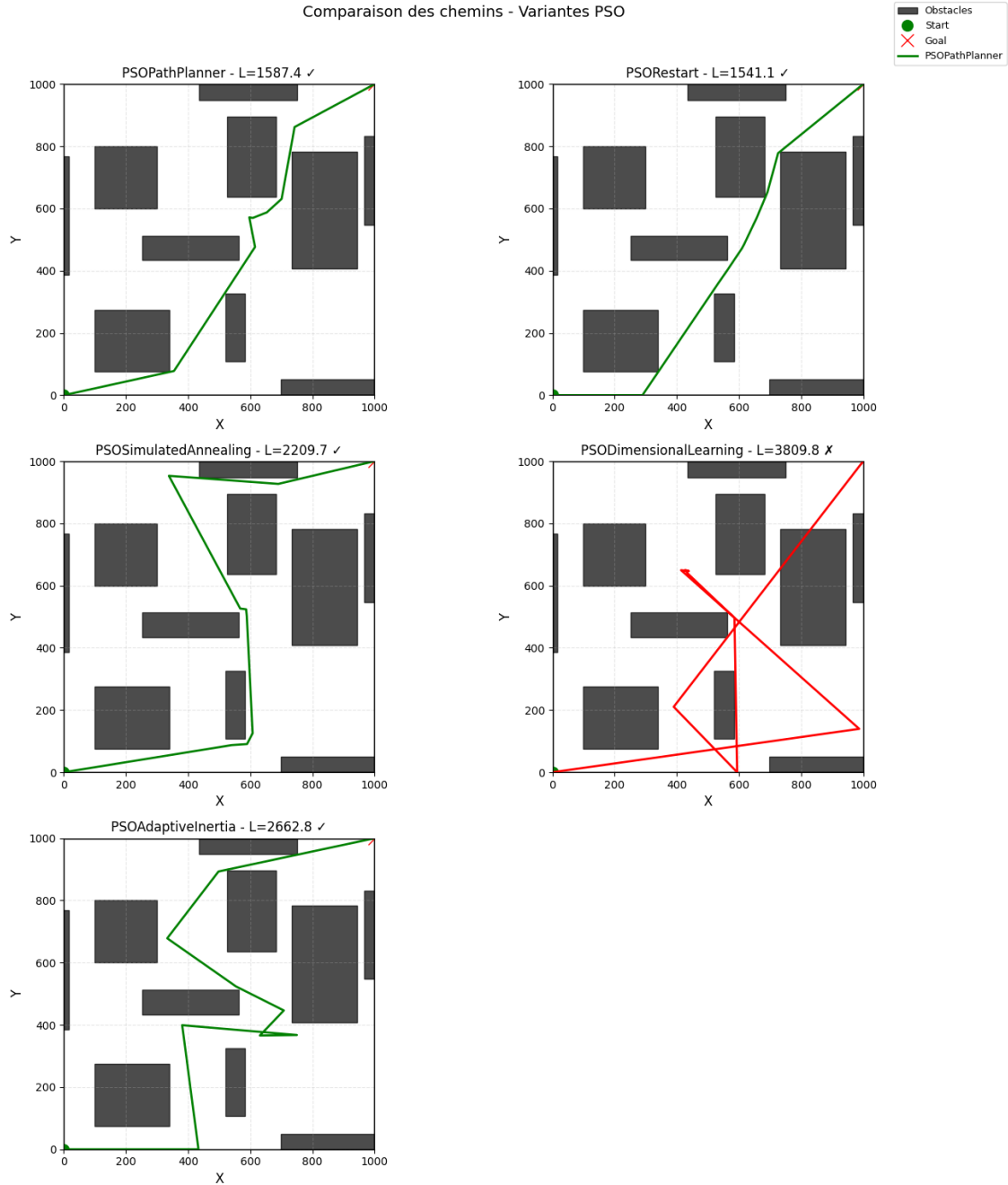
Base params: {'num_particles': 300, 'num_waypoints': 8, 'max_iter': 100, 'w': 0.8, 'c1': 1.6, 'c2': 1.0}

=====

[22]: *# Visualisation des chemins pour chaque variante*

```
fig = benchmark.plot_results_grid(
    results=pso_variant_results,
    envs={name: env for name in pso_variant_results.keys()},
    n_cols=2,
)
plt.suptitle("Comparaison des chemins - Variantes PSO", fontsize=14, y=1.02)
plt.tight_layout()
plt.show()
```

Comparaison des chemins - Variantes PSO



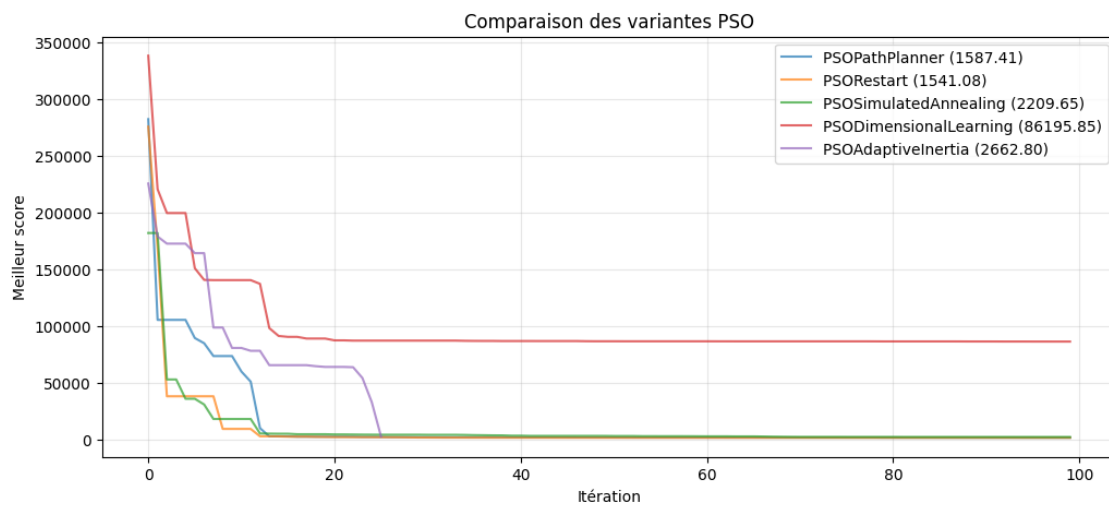
```
[23]: # Comparaison des variantes PSO - Convergence
plt.figure(figsize=(12, 5))
for name, result in pso_variant_results.items():
    if result.history:
        label = f"{name} ({result.score:.2f})" if result.score else name
        plt.plot(result.history, label=label, alpha=0.7)
```

```

plt.xlabel('Itération')
plt.ylabel('Meilleur score')
plt.title('Comparaison des variantes PSO')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Récapitulatif des performances PSO (utilisant display_results)
benchmark.display_results(
    list(pso_variant_results.values()),
    title="Comparaison des variantes PSO",
    sort_by='length'
)

```



Comparaison des variantes PSO

Algorithm	Score	Length	Valid	Iter	Time (s)	
↪						↪
-----						↪
↪ PSORestart	1541.08	1541.08		100	17.6875	↪
↪ PSOPathPlanner	1587.41	1587.41		100	17.6406	↪
↪ PSOSimulatedAnnealing	2209.65	2209.65		100	28.3125	↪
↪ PSOAdaptiveInertia	2662.80	2662.80		26	8.2188	↪
↪ PSODimensionalLearning	86195.85	3809.77		100	35.8750	↪
↪						

1.3 Partie 4 : Rapidly-exploring Random Tree (RRT*)

1.3.1 Question 13 : Structure de données pour l'arbre

```
class Node:
    x: float          # Coordonnée X
    y: float          # Coordonnée Y
    parent: Node      # Nœud parent (None pour la racine)
    cost: float       # Coût cumulé depuis la racine
```

Arbre : Liste de nœuds List[Node]

1.3.2 Question 14 : Reconstruction du chemin

```
Fonction ReconstructPath(v):
    chemin = []
    Tant que v != Null :
        chemin.ajouter(v.position)
        v = v.parent
    Retourner chemin.inverser()
```

Complexité : $O(h)$ où h est la profondeur du nœud.

1.3.3 Question 15 : Pseudo-code RRT*

Algorithme RRT_Star(Start, Goal, max_iter, _s, _r, goal_bias)

Entrées:

Start : Point de départ
Goal : Point d'arrivée
max_iter : Nombre maximal d'itérations
_s (delta_s) : Distance maximale de steering (extension)
_r (delta_r) : Rayon de recherche des voisins pour rewiring
goal_bias : Probabilité de choisir Goal comme cible (ex: 0.05) ; permet de rendre la recherche plus efficace

// Initialisation

Arbre = [Node(Start, parent=None, cost=0)]

// Boucle principale

Pour k = 1 à max_iter :

 // Échantillonnage biaisé vers le goal

 r = nombre aléatoire dans [0, 1]

 Si r < goal_bias :

 v_rand = Goal

 // Avec proba goal_bias, on cible le goal

```

Sinon :
    v_rand = point aléatoire uniforme dans l'espace

// Trouver le nœud le plus proche
v_nearest = argmin_{v dans Arbre} distance(v, v_rand)

// Steering : créer un nouveau point vers v_rand
direction = (v_rand - v_nearest) / ||v_rand - v_nearest||
distance_to_rand = min(||v_rand - v_nearest||, _s)
v_new = v_nearest + direction * distance_to_rand

// Vérification collision du segment
Si segment(v_nearest, v_new) sans collision :

    // Choose Best Parent : chercher le meilleur parent parmi les voisins
    voisins = {v Arbre : distance(v, v_new) _r}
    best_parent = v_nearest
    best_cost = v_nearest.cost + distance(v_nearest, v_new)

    Pour chaque v voisins :
        Si segment(v, v_new) sans collision :
            cost_via_v = v.cost + distance(v, v_new)
            Si cost_via_v < best_cost :
                best_parent = v
                best_cost = cost_via_v

    // Ajouter v_new à l'arbre
    v_new.parent = best_parent
    v_new.cost = best_cost
    Arbre.ajouter(v_new)

// Rewiring : améliorer les voisins si v_new offre un raccourci
Pour chaque v voisins, v best_parent :
    Si segment(v_new, v) sans collision :
        new_cost = v_new.cost + distance(v_new, v)
        Si new_cost < v.cost :
            v.parent = v_new
            v.cost = new_cost

// Reconstruction du chemin
Retourner ReconstructPath(nœud le plus proche de Goal)

```

1.3.4 Question 16 : Complexité d'une itération RRT*

Soit n le nombre de nœuds actuellement dans l'arbre, M le nombre d'obstacles.

Détail par étape :

Étape	Opération	Complexité
1. Échantillonnage	Générer un point aléatoire	$O(1)$
2. Nearest Neighbor	Trouver le nœud le plus proche (parcours naïf)	$O(n)$
3. Steering	Calculer v_new (opérations vectorielles)	$O(1)$
4. Collision segment	Tester collision avec M obstacles	$O(M)$
5. Find Neighbors	Trouver voisins dans rayon δ_r	$O(n)$
6. Choose Best Parent	Tester collision pour chaque voisin (pire cas : tous)	$O(n \cdot M)$
7. Rewiring	Tester et mettre à jour les voisins	$O(n \cdot M)$

Complexité totale d'une itération :

$$O(n \cdot M)$$

- Le terme dominant est le **rewiring** et le **choose best parent**, car on teste potentiellement la collision de $O(n)$ segments avec M obstacles.

Complexité totale sur K itérations : À l'itération k , l'arbre contient au plus k nœuds, donc :

$$\sum_{k=1}^K O(k \cdot M) = O(M) \cdot \sum_{k=1}^K k = O(M \cdot K^2)$$

$$O(K^2 \cdot M)$$

1.3.5 Question 17 : Implémentation

```
[ ]: from random_explorer.rrt import RRTPlanner

scenario = SCENARIOS["scenario4"]
env = Environment(scenario)

rrt = RRTPlanner(
    env,
    max_iter=2000,
    delta_s=200.0,
    delta_r=200.0,
    goal_bias=0.05,
    goal_tolerance=20.0
)

rrt_path, rrt_length, _ = rrt.solve()

if rrt_path:
```

```

    print(f"RRT* - Chemin trouvé! Longueur: {rrt_length:.2f}, Nœuds: {len(rrt.
↪nodes)}")
else:
    print("RRT* - Pas de chemin trouvé")

```

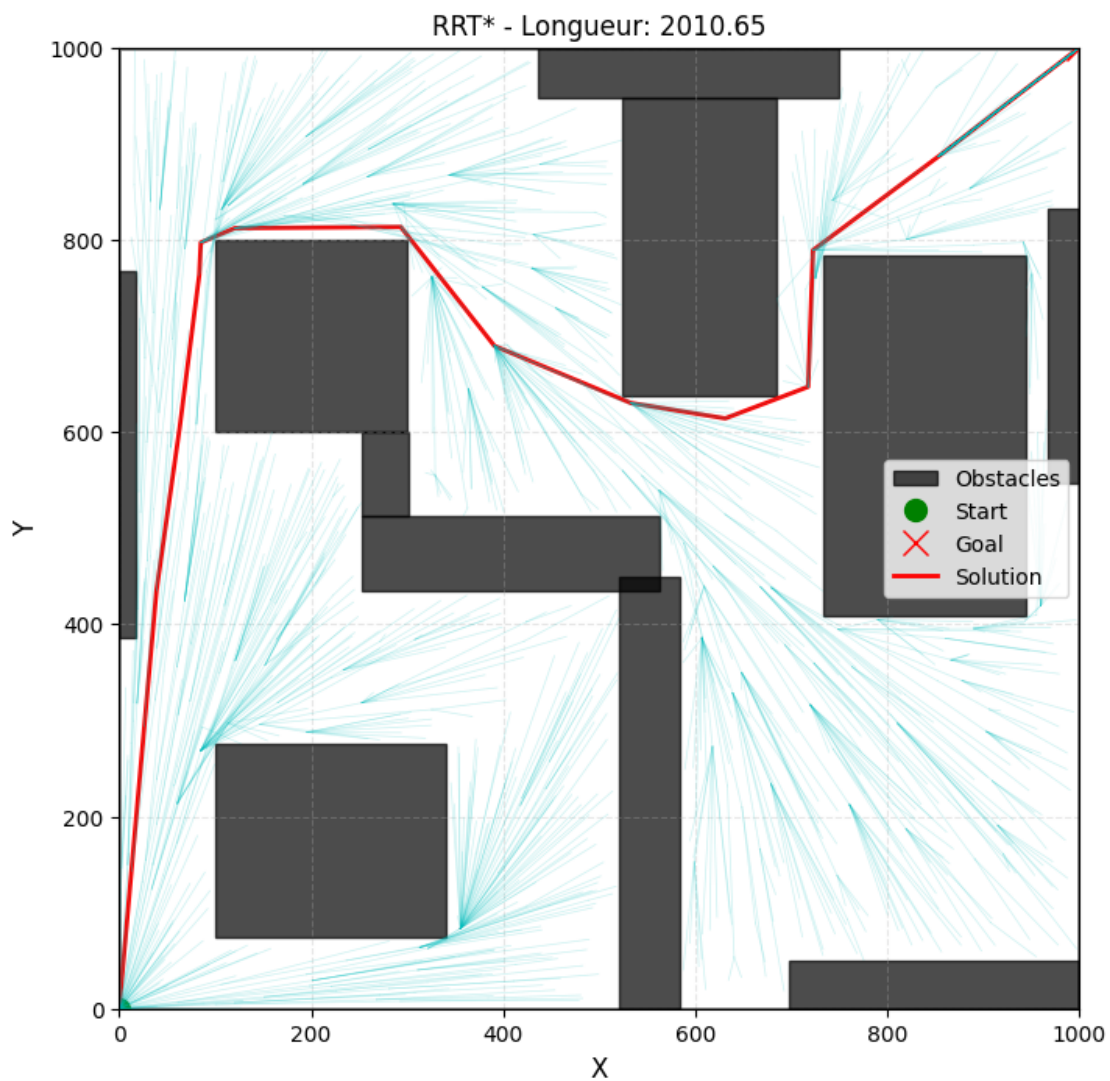
RRT*: 100%| | 2000/2000 [00:12<00:00, 154.08it/s, best_cost=2010.7]

RRT* - Chemin trouvé! Longueur: 2010.65, Nœuds: 1264

```

[25]: fig, ax = rrt.plot_tree(path=rrt_path, title=f"RRT* - Longueur: {rrt_length:.
↪2f}")
plt.show()

```



1.3.6 Question 18 : Performances sur tous les scénarios

```
[26]: RRT_PARAMS = {
    'max_iter': 2000,
    'delta_s': 200.0,
    'delta_r': 200.0,
    'goal_bias': 0.05
}

print(f"Benchmark RRT* sur tous les scénarios (max_iter = {RRT_PARAMS['max_iter']})")
print("="*60)

rrt_results = {}
for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)
    result = benchmark.run_rrt(env_test, False, False, RRT_PARAMS)
    result.scenario = name
    rrt_results[name] = result

    valid_str = " " if result.is_valid else " "
    length_str = f"{result.path_length:.2f}" if result.path_length < float('inf') else "N/A"
    print(f"{name}: Length={length_str}, Valid={valid_str}, Time={result.cpu_time:.3f}s")
```

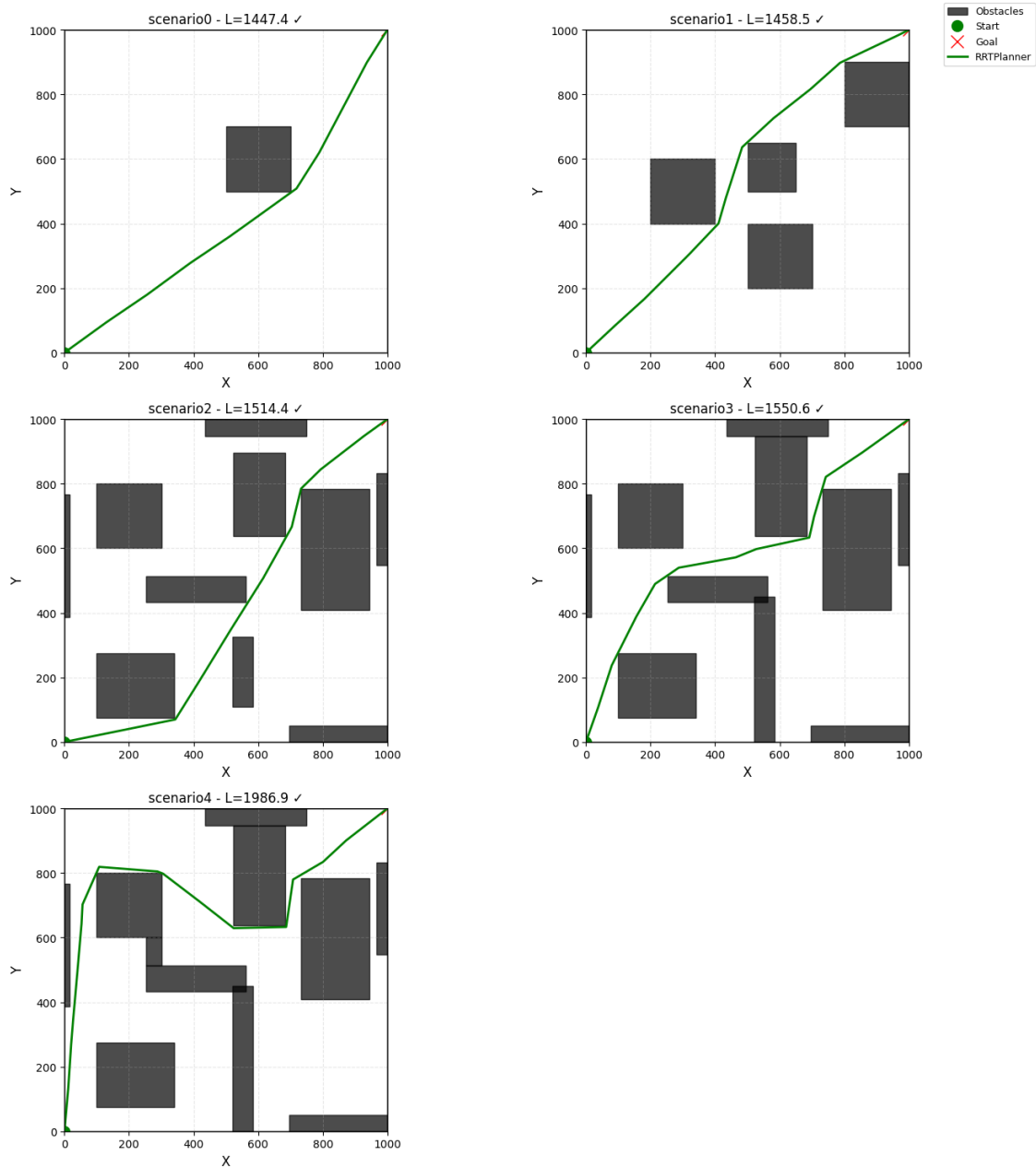
Benchmark RRT* sur tous les scénarios (max_iter = 2000)

=====

```
RRT*: 100%|      | 2000/2000 [00:19<00:00, 101.37it/s, best_cost=1447.4]
scenario0: Length=1447.42, Valid= , Time=13.781s
RRT*: 100%|      | 2000/2000 [00:16<00:00, 123.66it/s, best_cost=1458.5]
scenario1: Length=1458.47, Valid= , Time=10.031s
RRT*: 100%|      | 2000/2000 [00:10<00:00, 189.50it/s, best_cost=1514.4]
scenario2: Length=1514.44, Valid= , Time=7.625s
RRT*: 100%|      | 2000/2000 [00:09<00:00, 220.57it/s, best_cost=1550.6]
scenario3: Length=1550.57, Valid= , Time=7.031s
RRT*: 100%|      | 2000/2000 [00:05<00:00, 342.05it/s, best_cost=1986.9]
scenario4: Length=1986.88, Valid= , Time=4.922s
```

```
[27]: # Visualisation des chemins RRT pour tous les scénarios
envs = {name: Environment(path) for name, path in SCENARIOS.items()}
```

```
benchmark.plot_results_grid(rrt_results, envs, n_cols=2)
plt.show()
```



```
[28]: # Comparaison PSO vs RRT
print("\nComparaison PSO vs RRT*")
print("="*70)
print(f"{'Scénario':<12} {'PSO Length':>12} {'RRT Length':>12} {'PSO Time':>10}{'RRT Time':>10}")
```

```

print("-"*70)

for name in SCENARIOS.keys():
    pso_r = pso_results[name]
    rrt_r = rrt_results[name]
    pso_len = f"{pso_r.path_length:.2f}" if pso_r.path_length < float('inf')
    ↪ else "N/A"
    rrt_len = f"{rrt_r.path_length:.2f}" if rrt_r.path_length < float('inf')
    ↪ else "N/A"
    print(f"{name:<12} {pso_len:>12} {rrt_len:>12} {pso_r.cpu_time:>10.3f}
    ↪ {rrt_r.cpu_time:>10.3f}")

```

Comparaison PSO vs RRT*

Scénario	PSO Length	RRT Length	PSO Time	RRT Time
scenario0	1445.90	1447.42	1.625	13.781
scenario1	1965.97	1458.47	3.156	10.031
scenario2	1656.70	1514.44	6.438	7.625
scenario3	1654.73	1550.57	13.359	7.031
scenario4	2228.36	1986.88	9.594	4.922

Conclusion :

Aspect	PSO	RRT*
Fiabilité	Variable	Très fiable
Paramétrage	Plus complexe	Simple

RRT* est plus robuste pour les environnements complexes, PSO plus rapide sur les cas simples.

1.3.7 Question 19 : Analyse de complexité de l'optimisation de chemin

Notations : - n : nombre de points dans le chemin à optimiser - M : nombre d'obstacles dans l'environnement

Principe de l'algorithme d'optimisation L'algorithme parcourt le chemin et tente de "sauter" directement vers les points les plus éloignés possibles sans collision (inégalité triangulaire).

Analyse de complexité Boucle principale : - Dans le pire cas, on visite chaque point du chemin : $O(n)$ itérations

Recherche du point le plus loin atteignable : - Pour chaque point courant, on teste les connexions directes vers tous les points suivants - Dans le pire cas (aucun raccourci possible) : on teste $(n-1) + (n-2) + \dots + 1 = \frac{n(n-1)}{2}$ segments - Complexité de cette étape : $O(n^2)$

Test de collision par segment : - Chaque test `_check_collision()` utilise `evaluate_path_collision()` de l'environnement - Cette méthode teste l'intersection du segment avec chaque obstacle : $O(M)$ - Complexité par test de collision : $O(M)$

Calcul de la longueur du chemin optimisé : - Parcours de tous les points du chemin optimisé - Dans le meilleur cas (chemin très raccourci) : $O(1)$ à $O(\log n)$ - Dans le pire cas (aucune optimisation) : $O(n)$

Complexités Complexité par optimisation :

- **Meilleur cas :**
 - Peu de tests nécessaires, chemin rapidement simplifié
 - $O(n \cdot M)$
 - **Cas moyen :**
 - Quelques raccourcis trouvés, réduction modérée du nombre de tests
 - $O(n^2 \cdot M)$
 - **Pire cas** (environnement très contraint, aucun raccourci) :
 - Test de tous les segments possibles
 - $O(n^2 \cdot M)$
-

Complexité totale de l'algorithme RRT* avec optimisation Si on note : - K : nombre d'itérations RRT* - δ_r : rayon de recherche des voisins - n_{tree} : nombre de nœuds dans l'arbre (K en pratique) - n_{path} : nombre de points dans le chemin final

Sans optimisation :

$$O(K \cdot n_{\text{tree}} \cdot M)$$

Avec optimisation :

$$O(K \cdot n_{\text{tree}} \cdot M + n_{\text{path}}^2 \cdot M)$$

1.3.8 Question 20 : Test de l'optimisation de chemin

```
[29]: RRT_PARAMS = {
    'max_iter': 2000,
    'delta_s': 200.0,
    'delta_r': 200.0,
    'goal_bias': 0.05
}

print(f"Benchmark RRT* sur tous les scénarios (max_iter = {RRT_PARAMS['max_iter']})")
print("="*60)

rrt_opti_results = {}
for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)
    result = benchmark.run_rrt(env_test, True, False, RRT_PARAMS)
```

```

result.scenario = name
rrt_opti_results[name] = result

valid_str = " " if result.is_valid else " "
length_str = f"{result.path_length:.2f}" if result.path_length <
↳float('inf') else "N/A"
print(f"{name}: Length={length_str}, Valid={valid_str}, Time={result.
↳cpu_time:.3f}s")

```

Benchmark RRT* sur tous les scénarios (max_iter = 2000)

=====

RRT* optimized: 100%| | 2000/2000 [00:15<00:00, 126.08it/s,
best_cost=1444.8]

scenario0: Length=1444.06, Valid= , Time=11.266s

RRT* optimized: 100%| | 2000/2000 [00:15<00:00, 130.26it/s,
best_cost=1461.7]

scenario1: Length=1460.83, Valid= , Time=11.078s

RRT* optimized: 100%| | 2000/2000 [00:09<00:00, 214.63it/s,
best_cost=1525.5]

scenario2: Length=1524.20, Valid= , Time=7.234s

RRT* optimized: 100%| | 2000/2000 [00:09<00:00, 210.20it/s,
best_cost=1547.8]

scenario3: Length=1544.88, Valid= , Time=7.969s

RRT* optimized: 100%| | 2000/2000 [00:05<00:00, 376.45it/s,
best_cost=2022.3]

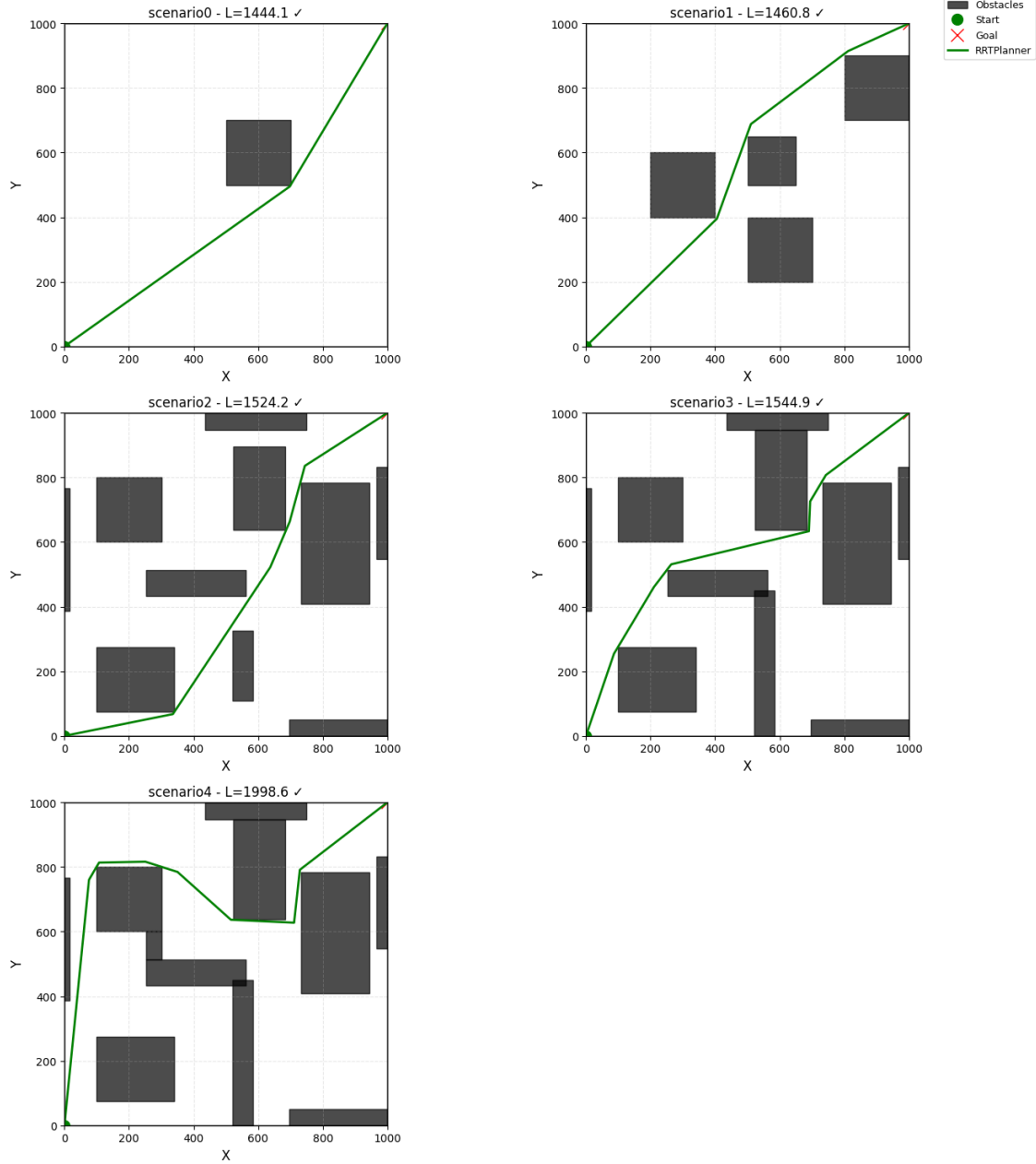
scenario4: Length=1998.62, Valid= , Time=4.203s

[30]:

```

envs = {name: Environment(path) for name, path in SCENARIOS.items()}
benchmark.plot_results_grid(rrt_opti_results, envs, n_cols=2)
plt.show()

```



```
[31]: print("\nComparaison PSO vs RRT* vs RRT* optim")
print("="*90)
print(f"{'Scénario':<12} {'PSO Length':>12} {'RRT Length':>12} {'RRT_opti_
Length':>12} {'PSO Time':>10} {'RRT Time':>10} {'RRT_opti Time':>10}")
print("-"*90)

for name in SCENARIOS.keys():
    pso_r = pso_results[name]
```

```

rrt_r = rrt_results[name]
rrt_opti_r = rrt_opti_results[name]
pso_len = f"{pso_r.path_length:.2f}" if pso_r.path_length < float('inf')
↳ else "N/A"
rrt_len = f"{rrt_r.path_length:.2f}" if rrt_r.path_length < float('inf')
↳ else "N/A"
rrt_opti_len = f"{rrt_opti_r.path_length:.2f}" if rrt_opti_r.path_length <
↳ float('inf') else "N/A"
print(f"{name:<12} {pso_len:>12} {rrt_len:>12} {rrt_opti_len:>12} {pso_r.
↳ cpu_time:>10.3f} {rrt_r.cpu_time:>10.3f} {rrt_opti_r.cpu_time:>10.3f}")

```

Comparaison PSO vs RRT* vs RRT* optim

```

=====
=====
Scénario      PSO Length   RRT Length RRT_opti Length   PSO Time   RRT Time
RRT_opti Time
-----
-----
scenario0      1445.90      1447.42      1444.06      1.625      13.781
11.266
scenario1      1965.97      1458.47      1460.83      3.156      10.031
11.078
scenario2      1656.70      1514.44      1524.20      6.438      7.625
7.234
scenario3      1654.73      1550.57      1544.88      13.359      7.031
7.969
scenario4      2228.36      1986.88      1998.62      9.594      4.922
4.203

```

```

[32]: RRT_PARAMS = {
    'max_iter': 2000,
    'delta_s': 300.0,
    'delta_r': 300.0,
    'goal_bias': 0.05
}

print(f"Benchmark RRT* sur tous les scénarios (max_iter =
↳ {RRT_PARAMS['max_iter']})")
print("="*60)

rrt_int_results = {}
for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)
    result = benchmark.run_rrt(env_test, False, True, RRT_PARAMS)
    result.scenario = name
    rrt_int_results[name] = result

```

```

    valid_str = " " if result.is_valid else " "
    length_str = f"{result.path_length:.2f}" if result.path_length <
↳float('inf') else "N/A"
    print(f"{name}: Length={length_str}, Valid={valid_str}, Time={result.
↳cpu_time:.3f}s")

```

Benchmark RRT* sur tous les scénarios (max_iter = 2000)

=====

RRT* intelligent: 100%| | 2000/2000 [01:53<00:00, 17.60it/s]

scenario0: Length=N/A, Valid= , Time=91.094s

RRT* intelligent: 100%| | 2000/2000 [00:45<00:00, 43.92it/s]

scenario1: Length=1555.02, Valid= , Time=37.203s

RRT* intelligent: 100%| | 2000/2000 [00:25<00:00, 77.00it/s]

scenario2: Length=1550.13, Valid= , Time=21.250s

RRT* intelligent: 100%| | 2000/2000 [00:14<00:00, 139.39it/s]

scenario3: Length=1612.36, Valid= , Time=13.125s

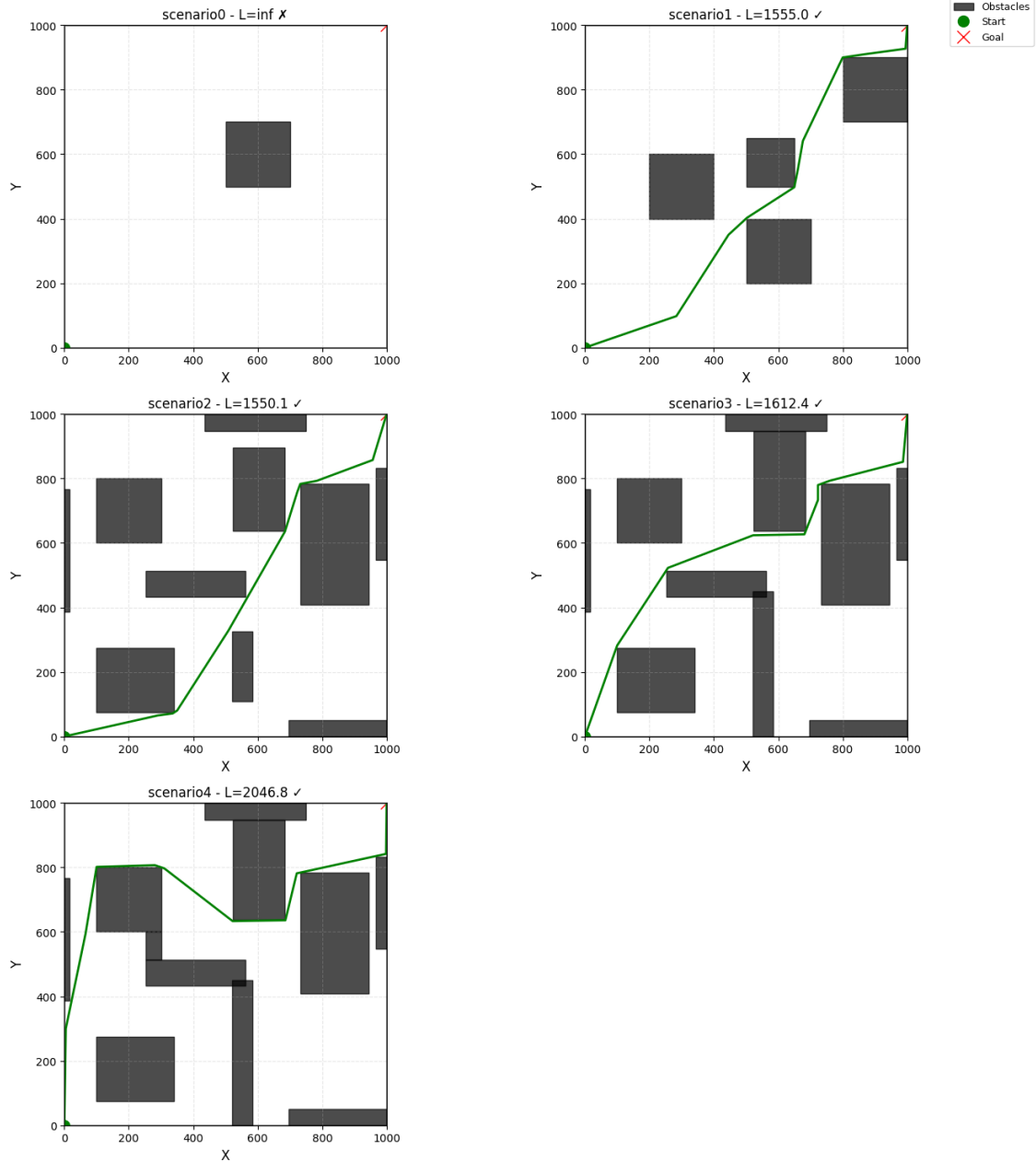
RRT* intelligent: 100%| | 2000/2000 [00:11<00:00, 171.73it/s]

scenario4: Length=2046.76, Valid= , Time=10.922s

```

[33]: envs = {name: Environment(path) for name, path in SCENARIOS.items()}
      benchmark.plot_results_grid(rrt_int_results, envs, n_cols=2)
      plt.show()

```



1.3.9 Question 21 & 22 : Méthode d'échantillonnage intelligent & Tests

```
[57]: RRT_PARAMS = {
    'max_iter': 2000,
    'delta_s': 200.0,
    'delta_r': 200.0,
    'goal_bias': 0.05
}
```

```

print(f"Benchmark RRT* sur tous les scénarios (max_iter =
↳{RRT_PARAMS["max_iter"]})")
print("="*60)

rrt_opti_int_results = {}
for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)
    result = benchmark.run_rrt(env_test, True, True, RRT_PARAMS)
    result.scenario = name
    rrt_opti_int_results[name] = result

    valid_str = " " if result.is_valid else " "
    length_str = f"{result.path_length:.2f}" if result.path_length <
↳float('inf') else "N/A"
    print(f"{name}: Length={length_str}, Valid={valid_str}, Time={result.
↳cpu_time:.3f}s")

```

Benchmark RRT* sur tous les scénarios (max_iter = 2000)

=====

```

RRT* intelligent optimized: 100%|      | 2000/2000 [01:03<00:00, 31.40it/s]
scenario0: Length=N/A, Valid= , Time=49.141s
RRT* intelligent optimized: 100%|      | 2000/2000 [00:30<00:00, 65.00it/s]
scenario1: Length=1449.73, Valid= , Time=23.438s
RRT* intelligent optimized: 100%|      | 2000/2000 [00:16<00:00, 119.29it/s]
scenario2: Length=1511.05, Valid= , Time=14.266s
RRT* intelligent optimized: 100%|      | 2000/2000 [00:07<00:00, 271.99it/s]
scenario3: Length=1531.38, Valid= , Time=6.062s
RRT* intelligent optimized: 100%|      | 2000/2000 [00:07<00:00, 278.03it/s]
scenario4: Length=1966.07, Valid= , Time=5.781s

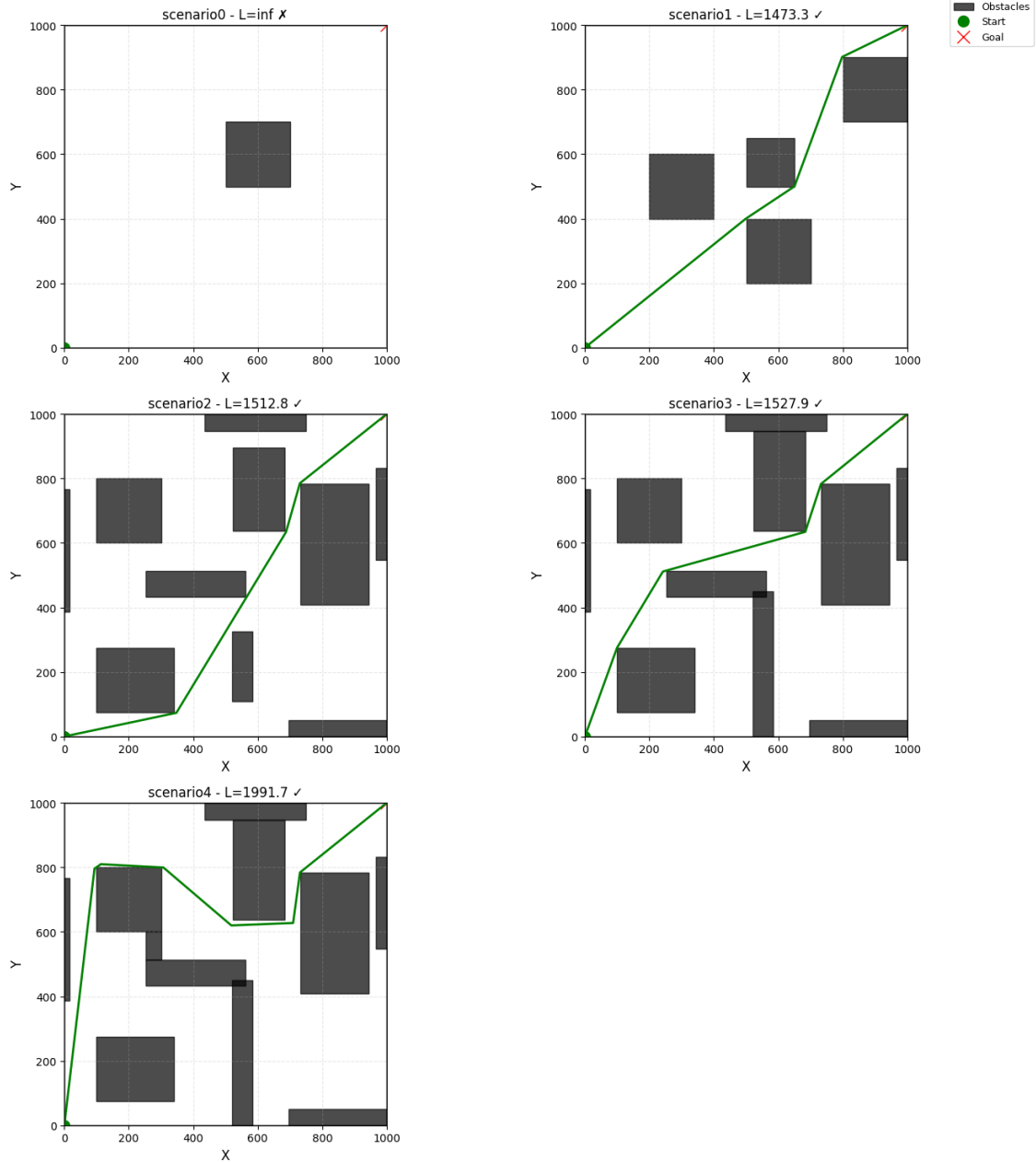
```

[35]:

```

envs = {name: Environment(path) for name, path in SCENARIOS.items()}
benchmark.plot_results_grid(rrt_opti_int_results, envs, n_cols=2)
plt.show()

```



```
[36]: print("\nComparaison RRT* vs RRT* optim vs RRT* opti int")
print("="*110)
print(f"{'Scénario':<12} {'RRT Length':>12} {'RRT_op Length':>14} {'RRT_op_int_␣
↳Length':>14} {'RRT Time':>14} {'RRT_op Time':>14} {'RRT_op_int Time':>14}")
print("-"*110)

for name in SCENARIOS.keys():
    rrt_r = rrt_results[name]
```

```

rrt_opti_r = rrt_opti_results[name]
rrt_opti_int_r = rrt_opti_int_results[name]
rrt_len = f"{rrt_r.path_length:.2f}" if rrt_r.path_length < float('inf')
↪ else "N/A"
rrt_opti_len = f"{rrt_opti_r.path_length:.2f}" if rrt_opti_r.path_length <
↪ float('inf') else "N/A"
rrt_opti_int_len = f"{rrt_opti_int_r.path_length:.2f}" if rrt_opti_int_r.
↪ path_length < float('inf') else "N/A"
print(f"{name:<12} {rrt_len:>12} {rrt_opti_len:>14} {rrt_opti_int_len:>14}
↪ {rrt_r.cpu_time:>14.3f} {rrt_opti_r.cpu_time:>14.3f} {rrt_opti_int_r.
↪ cpu_time:>14.3f}")

```

Comparaison RRT* vs RRT* optim vs RRT* opti int

Scénario	RRT Length	RRT_op Length	RRT_op_int Length	RRT Time
RRT_op Time	RRT_op_int Time			
scenario0	1447.42	1444.06	N/A	13.781
11.266	39.891			
scenario1	1458.47	1460.83	1473.34	10.031
11.078	25.312			
scenario2	1514.44	1524.20	1512.77	7.625
7.234	12.500			
scenario3	1550.57	1544.88	1527.89	7.031
7.969	12.469			
scenario4	1986.88	1998.62	1991.69	4.922
4.203	4.078			

Question 23 Principe de l'algorithme

Afin de résoudre le problème lorsqu'il y a 2 robots, nous considérons le disque centré en un robot et de rayon R comme étant un obstacle pour l'autre robot. Notre proposition est de faire une planification alternée d'un robot ensuite de l'autre :

1. **Planifier pour le robot 1** : Créer un environnement temporaire où le robot 2 (à sa position actuelle) est un obstacle circulaire de rayon R
2. **Planifier pour le robot 2** : Créer un environnement temporaire où le robot 1 (à sa position actuelle) est un obstacle circulaire de rayon R
3. **Vérifier les collisions** : Comparer les positions des deux robots à chaque étape de temps (en supposant qu'ils se déplacent à vitesse constante)
4. **Itérer** : Si collision détectée, mettre à jour les positions de référence et replanifier

Pseudo-code

ALGORITHME Multi-Robot RRT*

```

ENTRÉES: env (start, goal, start2, goal2, radius), max_alternations

FONCTION solve_alternating():
    pos1 ← start
    pos2 ← start2

    POUR iteration de 1 à max_alternations:
        # Planifier pour robot 1 (robot 2 comme obstacle)
        env_temp1 ← créer_env_temporaire(robot=1, obstacle_circulaire=pos2)
        planner1 ← RRTPlanner(env_temp1, intelligent_sampling=True)
        path1 ← planner1.solve(optimized=True)

        SI path1 est None: RETOURNER échec

        # Planifier pour robot 2 (robot 1 comme obstacle)
        env_temp2 ← créer_env_temporaire(robot=2, obstacle_circulaire=pos1)
        planner2 ← RRTPlanner(env_temp2, intelligent_sampling=True)
        path2 ← planner2.solve(optimized=True)

        SI path2 est None: RETOURNER échec

        # Vérifier collision inter-robots
        SI NON check_inter_robot_collision(path1, path2):
            RETOURNER (path1, path2) # Succès

        # Mise à jour pour forcer replanning
        pos1 ← milieu(path1)
        pos2 ← milieu(path2)

    RETOURNER (path1, path2) # Meilleur trouvé

FONCTION créer_env_temporaire(robot, obstacle_circulaire):
    env_temp ← copie(env)

    # Approximer cercle par carré de côté 2*R
    x_obs ← obstacle_circulaire.x - radius
    y_obs ← obstacle_circulaire.y - radius
    env_temp.obstacles.append((x_obs, y_obs, 2*radius, 2*radius))

    SI robot == 1:
        env_temp.start ← start
        env_temp.goal ← goal
    SINON:
        env_temp.start ← start2
        env_temp.goal ← goal2

    RETOURNER env_temp

```

```

FONCTION check_inter_robot_collision(path1, path2):
    min_len ← min(longueur(path1), longueur(path2))

    POUR i de 0 à min_len:
        SI distance(path1[i], path2[i]) < 2*radius:
            RETOURNER Vrai

    RETOURNER Faux

```

Complexité

Par itération alternée : - Planification robot 1 : $O(K \cdot |V| \cdot (M + 1))$ - Planification robot 2 : $O(K \cdot |V| \cdot (M + 1))$ - Vérification collision : $O(\min(n, n))$

Complexité totale : $O(A \cdot K \cdot |V| \cdot M)$

où : - A = nombre d'alternations (max_alternations) - K = nombre d'itérations RRT* - $|V|$ = nombre de nœuds dans l'arbre - M = nombre d'obstacles - n, n = longueurs des chemins

1.3.10 Question 24: Scénarios de tests

```

[78]: %load_ext autoreload
      %autoreload 2

```

The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```

[ ]: from random_explorer.rrt import MultiRobotRRTPlanner

MULTI_RRT_PARAMS = {
    'max_iter': 1000,
    'delta_s': 200.0,
    'delta_r': 150.0,
    'goal_bias': 0.05,
    'goal_tolerance': 10.0
}

print(f"\nBenchmark Multi-Robot RRT* sur tous les scénarios")
print(f"max_iter = {MULTI_RRT_PARAMS['max_iter']}, R = {env.radius}")
print("="*90)

multi_robot_results = {}

for name, scenario_path in SCENARIOS.items():
    env_test = Environment(scenario_path)

    result = benchmark.run_multi_robot_rrt(
        env_test,
        optimized=True,
        intelligent=True,

```

```

        params=MULTI_RRT_PARAMS
    )

    result['scenario'] = name
    multi_robot_results[name] = result

    status = " " if result['success'] else " "
    len1 = f"{result['length1']:.2f}" if result['length1'] < float('inf') else
↪ "N/A"
    len2 = f"{result['length2']:.2f}" if result['length2'] < float('inf') else
↪ "N/A"

    print(f"\n{name}")

    print(f"Status: {status}")
    if result['success']:
        print(f"Robot 1: Length={len1}, Iter={result['iterations1']}")
        print(f"Robot 2: Length={len2}, Iter={result['iterations2']}")
        print(f"CPU Time: {result['cpu_time']:.3f}s")
    else:
        print(f"Failed to find solution")

```

Benchmark Multi-Robot RRT* sur tous les scénarios

max_iter = 1000, R = 50.0

=====

RRT* intelligent optimized: 100%| | 1000/1000 [00:09<00:00, 110.81it/s]
Multi-Robot RRT* intelligent otpimized: 0%| | 0/1000 [00:09<?, ?it/s,
Robot 1 failed]

scenario0

Status:

Failed to find solution

RRT* intelligent optimized: 100%| | 1000/1000 [00:04<00:00, 225.44it/s]
RRT* intelligent optimized: 100%| | 1000/1000 [00:03<00:00, 259.37it/s]
Multi-Robot RRT* intelligent otpimized: 0%| | 0/1000 [00:08<?, ?it/s,
Success! L1=1479.7, L2=1059.5]

scenario1

Status:

Robot 1: Length=1479.68, Iter=1000

Robot 2: Length=1059.48, Iter=1000

CPU Time: 7.141s

RRT* intelligent optimized: 100%| | 1000/1000 [00:02<00:00, 360.45it/s]

```
RRT* intelligent optimized: 100%|      | 1000/1000 [00:02<00:00, 369.32it/s]
Multi-Robot RRT* intelligent otpimized: 0%|      | 0/1000 [00:05<?, ?it/s,
Success! L1=1519.0, L2=1049.2]
```

scenario2

Status:

Robot 1: Length=1518.99, Iter=1000

Robot 2: Length=1049.19, Iter=1000

CPU Time: 4.734s

```
RRT* intelligent optimized: 100%|      | 1000/1000 [00:02<00:00, 399.08it/s]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:02<00:00, 427.05it/s]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 524.76it/s]
4.88s/it, L1=1921.0, L2=1219.9, I1=1000, I2=1000, coll=Y]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 502.86it/s]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 534.94it/s]
4.32s/it, L1=1532.4, L2=1418.3, I1=1000, I2=1000, coll=Y]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 526.35it/s]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 637.03it/s]
4.08s/it, L1=1535.9, L2=1252.3, I1=1000, I2=1000, coll=Y]
Multi-Robot RRT* intelligent otpimized: 0%|      | 3/1000 [00:14<1:18:31,
4.73s/it, Robot 1 failed]
```

scenario3

Status:

Failed to find solution

```
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 521.08it/s]
RRT* intelligent optimized: 100%|      | 1000/1000 [00:01<00:00, 983.65it/s]
Multi-Robot RRT* intelligent otpimized: 0%|      | 0/1000 [00:02<?, ?it/s,
Robot 2 failed]
```

scenario4

Status:

Failed to find solution

```
[73]: print("\n\nRécapitulatif Multi-Robot RRT*")
print("="*110)
print(f"{'Scénario':<12} {'Status':<8} {'R1 Length':>12} {'R2 Length':>12} ␣
↳{'R1 Iter':>10} {'R2 Iter':>10} {'Time (s)':>10}")
print("-"*110)

for name in SCENARIOS.keys():
    result = multi_robot_results[name]
    status = "Succès" if result['success'] else "Échec"
```

```

len1 = f"{result['length1']:.2f}" if result['length1'] < float('inf') else
↳ "N/A"
len2 = f"{result['length2']:.2f}" if result['length2'] < float('inf') else
↳ "N/A"
iter1 = f"{result['iterations1']}" if result['success'] else "N/A"
iter2 = f"{result['iterations2']}" if result['success'] else "N/A"
time_str = f"{result['cpu_time']:.3f}" if result['success'] else "N/A"

print(f"{name:<12} {status:<8} {len1:>12} {len2:>12} {iter1:>10}
↳ {iter2:>10} {time_str:>10}")

```

Récapitulatif Multi-Robot RRT*

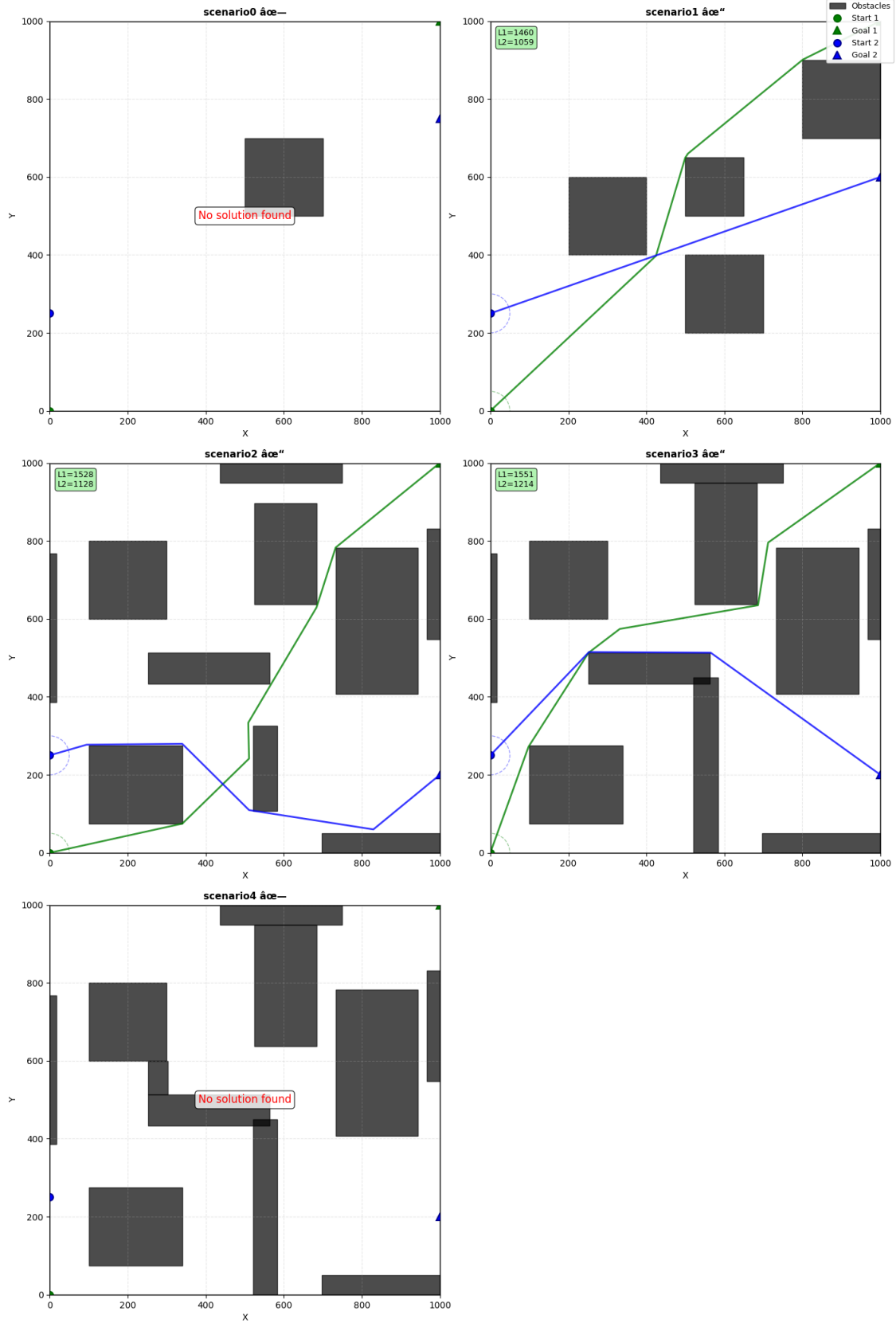
Scénario	Status	R1 Length	R2 Length	R1 Iter	R2 Iter
Time (s)					
scenario0	Échec	N/A	N/A	N/A	N/A
N/A					
scenario1	Succès	1477.43	1059.48	1000	1000
12.375					
scenario2	Succès	1510.58	1205.57	1000	1000
7.000					
scenario3	Succès	1535.24	1219.23	1000	1000
4.703					
scenario4	Échec	N/A	N/A	N/A	N/A
N/A					

```

[79]: envs = {name: Environment(path) for name, path in SCENARIOS.items()}

benchmark.plot_multi_robot_results_grid(
    multi_robot_results,
    envs,
    n_cols=2,
    show_safety_zones=True
)
plt.show()

```



```
[ ]: env = Environment("data/scenario1.txt")

print(f"Start: {env.start}, in collision: {env.is_collision(env.start)}")
print(f"Goal: {env.goal}, in collision: {env.is_collision(env.goal)}")

planner = RRTPlanner(
    env=env,
    max_iter=2000,
    delta_s=200.0,
    delta_r=200.0,
    goal_bias=0.05,
    goal_tolerance=10.0
)

path, length, iters = planner.solve(optimized=True, intelligent_sampling=True,
    ↪show_progress=True)

print(f"\nResult: {path is not None}, Length: {length}, Iterations: {iters}")
```

Start: (0.0, 0.0), in collision: False

Goal: (1000.0, 1000.0), in collision: False

RRT* intelligent optimized: 100%| | 2000/2000 [00:31<00:00, 64.45it/s]

Result: True, Length: 1472.9060742734225, Iterations: 2000